Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Távközlési és Mesterséges Intelligencia Tanszék

Bacsó Janka Édua

ONLINE KÁRTYAJÁTÉK FEJLESZTÉSE MODERN WEBES TECHNOLÓGIÁKKAL

KONZULENS

Frankó Attila Ernő

BUDAPEST, 2025

Tartalomjegyzék

Tartalomjegyzék **2**
Összefoglaló **5**
Abstract **6**
1 Bevezetés **7**
2 Tervezés **8**
2.1 Feladat részletes leírása 8
2.1.1 Felhasználói követelmények ismertetése 8
2.1.2 Rendszer követelmények ismertetése 9
2.1.3 Játékmenet és szabályok részletes leírása 9
2.2 Funkcionális Követelmények 10
2.3 Rendszerterv 13
2.4 Tervezett technológiák 13
3 Technológiák **15**
3.1 Vue SPA + Vite 15
3.2 Go Programing Language (Golang, Go) 16
3.3 Cloud native patterns 17
3.3.1 Skálázhatóság 17
3.3.2 Lazán csatolt felépítés (Loose Coupling) 18
3.3.3 Ellenállóképesség 18
3.3.4 Menedzselhetőség 18
3.3.5 Megfigyelhetőség 19
3.3.6 Konténerizáció 19
3.4 Valós Idejű Kommunikáció 20
4 Implementáció **22**
4.1 Webalkalmazás felépítése 22
4.1.1 Frontend 22
4.1.2 Backend 28
4.1.3 Adatbázis 30
4.2 Lényegesebb funkciók 32
4.2.1 HTTPS 32
4.2.2 Authentikáció: 33

4.2.3 WebSocketen keresztül küldött üzenet kezelés 35
4.3 Docker és Docker Compose 38
4.3.1 A Docker architektúra 38
4.3.2 Dockerfileok 39
4.3.3 Docker Compose felépítés 41
5 Eredmény 45
5.1 Megvalósított Cloud Native Patternek 45
5.1.1 Konténerizáció 45
5.1.2 Állapotmentesség 45
5.1.3 Külső konfiguráció 46
5.1.4 Health checks 46
5.1.5 Graceful shutdown 47
5.1.6 Adatbázis interfész 48
5.2 Tovább fejlesztési lehetőségek 49
5.2.1 Redis 49
5.2.2 Logging 49
5.3 Konklúzió 50
6 Irodalomjegyzék 51
7 Függelék 52
7.1 I. Függelék Nyilatkozat generatív mesterséges intelligencia alkalmazásáról. 52

HALLGATÓI NYILATKOZAT

Alulírott **Bacsó Janka Édua**, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem. A Függelékben egyértelműen megjelöltem, hogy a mesterséges intelligencia eszközeit alkalmaztam-e a dolgozat elkészítéséhez; amennyiben igen, annak módját és mértékét a táblázatban közöltem. Tudomásul veszem, hogy a mesterséges intelligenciával generált tartalomért – annak mértékétől függetlenül – teljes felelősséggel tartozom.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK az interneten nyilvánosan hozzáférhetővé tegye, a munka teljes szövege pedig a BME Címtárban regisztrált személyek számára elérhető legyen. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Kelt: Budapest, 2025. 12. 21.

...……

Bacsó Janka Édua

Összefoglaló

A szakdolgozat célja egy online kártyajáték megtervezésének és megvalósításának bemutatása modern webtechnológiák alkalmazásával. A fejlesztett webalkalmazás az „Ulti” néven ismert, magyar kártyával játszott játék egy egyszerűsített, saját tovább gondolt változatára épül. A feladat a teljes rendszer megvalósítását magában foglalja, beleértve a frontend és backend komponensek tervezését és implementálását.

A dolgozat jelentős része az implementáció részletes bemutatására fókuszál. Ismertetésre kerülnek a fontosabb fejlesztési döntések, azok indoklása, valamint az alkalmazott technológiák konkrét használata a frontend és backend rétegekben

A webalkalmazás tervezése és fejlesztése során kiemelt hangsúlyt kapnak a cloud-native elvek és technológiák. A cél egy olyan alkalmazás bemutatása, amely megfelel a modern webes fejlesztési standardeknek, és képes rugalmasan alkalmazkodni a változó igényekhez. A dolgozat részletesen ismerteti azokat a módszereket és megoldásokat, amelyekkel ezt megszeretnénk valósítani.

Abstract

The aim of this thesis is to present the design and implementation of an online card game using modern web technologies. The developed web application is based on a simplified and independently reinterpreted version of *Ulti*, a traditional card game played with Hungarian playing cards. The task includes the implementation of the complete system, covering both the design and development of the frontend and backend components.

A significant portion of the thesis focuses on the detailed presentation of the implementation. It describes the key development decisions, their motivations, and the practical use of the selected technologies within the frontend and backend layers of the system.

During the design and development of the web application, special emphasis is placed on cloud-native principles and technologies. The goal is to present an application that complies with modern web development standards and is capable of adapting flexibly to changing requirements. The thesis provides a detailed description of the methods and solutions used to achieve these objectives.

1 Bevezetés

A mai internet világában bármilyen webalkalmazás fejlesztésébe kezdünk, elengedhetetlen, hogy megfelelően utánajárjunk annak, milyen eszközökkel és módszerekkel tudunk minél jobban alkalmazkodó applikációt készíteni. Az internet folyamatosan fejlődik, ezzel párhuzamosan pedig a felhasználók száma is növekszik, ezért már a tervezés korai szakaszában fontos figyelembe venni a skálázhatóságot és a jövőbeni bővíthetőséget. Egy jól megtervezett alkalmazás hosszú távon képes alkalmazkodni a változó igényekhez és terheléshez.

Az alábbi szakdolgozatban egy kevésbé ismert kártyajáték, az Ulti egy általam újragondolt változatának tervezését és megvalósítását mutatom be. Az Ulti magyar kártyával játszott, komplex szabályrendszerrel rendelkező játék. A megvalósítás során azonban olyan játék- és felhasználói felület kialakítására törekedtem, amely a kezdő játékosok számára is könnyen érthető és élvezhető, miközben lehetőséget hagy a későbbiekben egy összetettebb verzió bevezetésére.

A dolgozat célja egy egyszerűen használható, kellemes felhasználói élményt nyújtó online kártyajáték bemutatása, amelyen keresztül több modern webes technológia és fejlesztési megközelítés kerül ismertetésre. A bemutatott megoldások célja nem csupán a játék működésének megvalósítása, hanem annak szemléltetése is, hogy a megfelelő tervezési elvek alkalmazásával miként hozható létre egy bővíthető webalkalmazás.

2 Tervezés

Ebben a fejezetben a weboldal által teljesítendő követelményeket, elvárásokat írom le. Emellett egy rendszer tervet készítek ezek alapján.

2.1 Feladat részletes leírása

A webalkalmazás egy frontendből és egy backendből áll. A frontend a felhasználó által használt felület. Backend a szerver oldalát foglalja magába. Ezek mellett a backend kommunikál egy adatbázis kezelővel is.

2.1.1 Felhasználói követelmények ismertetése

A felhasználók az oldalt megnyitva tudnak elnavigálni a regisztráláshoz. Már fiókkal rendelkező felhasználók számára pedig elérhető a bejelentkezés opció.

Regisztráláshoz a felhasználónak meg kell adnia a nevét, email címét és egy jelszót. Belépésnél az email címre és jelszóra van szüksége a felhasználónak.

Bejelentkezést követően a felhasználó a főmenübe kerül. Itt lehetősége van belépni a játékközpontba, megnézni a profilját, vagy megnyitni a beállításokat. A felhasználó bármikor kijelentkezhet egy gomb kattintással.

Amikor a felhasználó belép a játékközpontba lehetősége van visszatérni vagy egy játékot indítani. Itt találja meg a játék leírását is. Jelenlegi verziójában csak egy játékmód érhető el a színjáték.

Ahhoz, hogy a játék elkezdődhessen szükségünk van 3 játékosra. Ha mind a 3 játékos belépett elindul a játék. Egy licitálás fázissal indul. A játékosok közül elő kell kerülni egy vállalónak, aki a másik két játékos ellen játszik. Ehhez a játékosok egy másra kontrázhatnak milyen színű játékot vállalnak be a többiekkel szemben. Ha eldőlt a vállaló szerepe, akkor számára elérhető opció lesz megnézni a talont és akár kicserélni a kártyait. Ezután indulhat a játék. A játékosok egymás után teszik lekártyáikat. Minden körben ki kerül egy nyertes, aki az új kört fogja indítani. A játék automatikusan véget ér mikor az utolsó játékos is letette a lapját. Ekkor a rendszer kiírja mindenki számára az eredményt. A játékosok a játék során bármikor dönthetnek úgy, hogy elhagyják a játékot. Ilyenkor minden fél számára megszakad a játék.

A játék végén lehetősége van a felhasználóknak arra, hogy visszalépjenek a játékközpontba vagy új játékot kezdjenek ugyanazokkal a játékosokkal.

A játékos a profil opcióra kattintva meg tudja tekinteni a fiókja adatait. A nevét, a mindenkori pont számát és lejátszott játékait. A játékokról részletes adat van, hogy kikkel játszott, ki mennyi pontot nyert/veszített stb.

A beállítások menüpontban a felhasználóak lehetősége van megváltoztatni a jelenlegi email címét vagy jelszavát.

2.1.2 Rendszer követelmények ismertetése

Weboldalunkon a felhasználók tudnak regisztrálni, majd bejelentkezés segítségével érik el annak a különböző funkcióit. A felhasználók nyilvántartásához egy adatbázis használunk melyben eltaroljuk a felhasználókat, e-mailjüket és a szerver által hash-elt jelszavukat.

Bejelentkezés során a felhasználok kapnak egy szerver által generált token-t. A token addig él amíg be vannak bejelentkezve vagy le nem jár. A token segítségével ellenőrizzük, hogy a beérkező kérés egy létező, állatalunk regisztrált felhasználótól ered és elvégezhető-e. Kijelentkezéskor ez a token törlődik. Egyszerre csak egy felhasználó lehet bejelentkezve egy adott fiókba.

A weboldal a legtöbb kérés egyszerű http kérésként szolgálja ki. A játékközpontba belépve viszont a szerver és felhasználó között erősebb kapcsolat/websocket connection létesül a zökkenő mentesebb kommunikáció érdekében. A szerver figyeli, hogy a kapcsolat nem szakad meg.

A profil oldalon a játékos látja előzőleg játszott meccseit és azok részletes adatait. Ezen adatokat szintén egy adatbázisban tároljuk, a fent említett adatbázistól eltérőben. Ezen adatok a következő: a játékosok, ki volt a bemondó, nyert-e a bemondó, hány pontot kapott a bemondó és hány pont a támadók, mikor volt a játék.

Felhasználónk mindig képes kijelentkezni az oldalról ilyenkor minden ideiglenes adat, amit a weboldal tárol róla megszűnik, beleértve a tokent is.

2.1.3 Játékmenet és szabályok részletes leírása

A játékhoz három játékosra van szükségünk. Magyar kártyával játszák, amiből minden játékos tíz darabot kap kézhez. A maradék két lap lesz a talon. A kétféle játékos

szerep van a bemondó és a támadó. Bemondóból csak egy lehet így értelemszerűen, aki nem bemondó az támadó. A játék egy úgynevezett „színjáték”, amiben a magyar kártya négy különböző szimbóluma közül kiválasztunk egyet adunak.

A szerepek a játék elején a licitálás fázisban kerülnek ki. Ezekben a körökben a játékosok a kezükben lévő lapok alapján licitálhatnak milyen „színjátékot” játszanak meg. Minél több azonos „szín” van egy játékos kezében annál nagyobb eséllyel tud, majd nyerni. A „színeknek” van erőssége és ha felül szeretnénk licitálni valakit akkor erősebb szint kell választanunk. Az erősség sorrendje a következő tök < makk < zöld < piros. A piros a legerősebb szín így, ha azt elértük a felül licitálásnál az adu színe nem változik csak a licit nő. Ha nem szeretnénk tétet tenni passzolhatunk. Az utolsó, aki licitált az lesz a bemondó.

Mielőtt a játék elindul a bemondónak lehetősége van megnézni a talont és kicserélni a kézben lévő kártyák közül bármire. Nem kötelező cserélni és nem kötelező megnézni a talont.

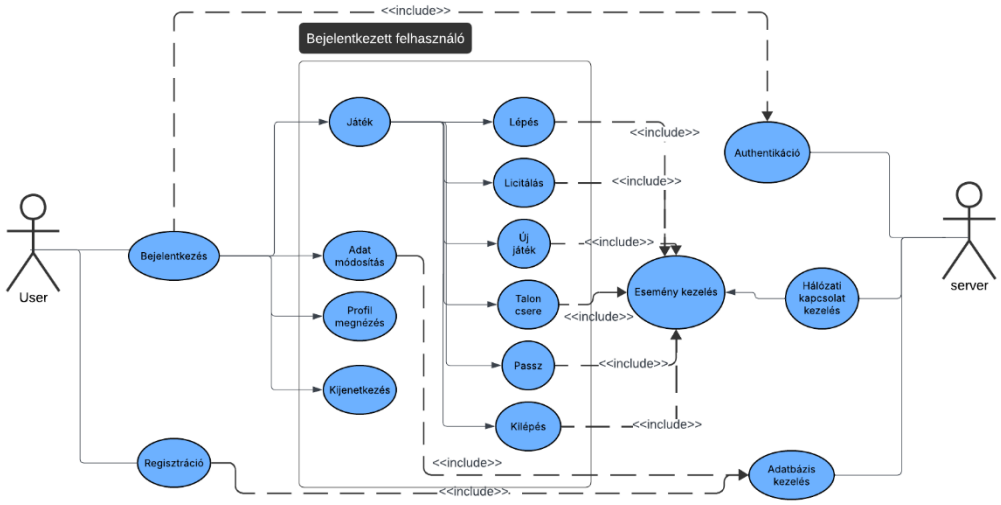
Egy játék tíz körből áll, és körönként mindenki egy lapot tesz le a kezéből. Körönként az a cél, hogy mi tegyük le a legtöbbet érő lapot ezzel ütve a többit. A kör első lapja adja meg a kör színét. Ha a játékosnak van a kezében olyan színű lap akkor kötelezően azt kell tennie, ha nincs akkor bármit tehet. Pontot akkor tud szerezni, ha az első lappal egyező színű, de nagyobb értékű lapot tesz le vagy ha adut. Az adu minden esetben viszi a kört kivéve, ha valaki nagyobb értékű adut tesz le. A lapok értéke a következő képen nő: hetes < nyolcas < kilences < tízes < alsó < felső < király < ász.

A játék célja, hogy minél több ütést szerezzünk. A bemondó akkor nyer, ha legalább 6 ütést szerzet. A támadók akkor nyernek, ha közösen szereznek legalább 5 ütést.

2.2 Funkcionális Követelmények

A webalkalmazásban két aktor különböztet meg a felhasználót (user/kliens) és a szervert. A diagram szemlélteti, hogy az egyes funkciók melyik aktorhoz kapcsolódnak, illetve hogyan oszlanak meg a felelősségi körök közöttük. Például a kliens az, aki a felületen keresztül regisztrál vagy bejelentkezik, míg az ezekhez tartozó autentikáció végrehajtását a szerveroldal valósítja meg.

A diagram továbbá azt is bemutatja, hogy a kliens számára bizonyos funkciók kizárólag bejelentkezett állapotban érhetők el. Ezek a funkciók a bejelentkezett felhasználóhoz tartozó Use Case-ekként jelennek meg.



1. ábra Alkalmazás Use Case Diagram

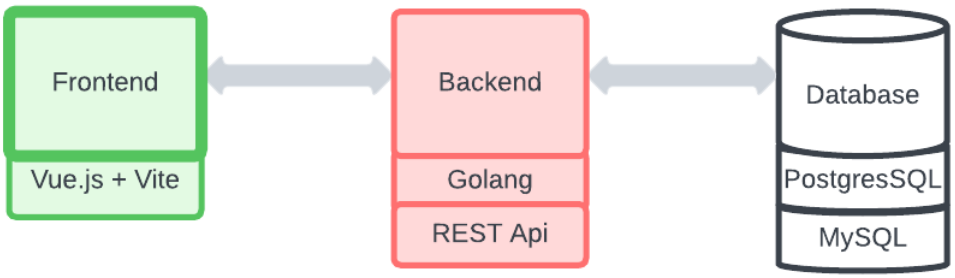
Az alábbi táblázatban szemléltetem az alkalmazás funkcionális követelményeit, valamint azt, hogy az egyes Use Case komponensek mely feladatok megvalósításáért felelősek.

Követelmény	**Use Case**
A felhasználó regisztrálhat az oldalra (nevet, email címet és jelszót megadva)	Regisztráció, Adatbázis kezelés
Csak regisztrált felhasználók tudnak bejelentkezni.	Authentikáció
A felhasználó bejelentkezhet fiókjában.	Bejelentkezés
Fiókkal rendelkező felhasználók elérik az oldal különböző funkcióit.	Bejelentkezés
Bejelentkezett felhasználok ki tudnak jelentkezni.	Kijelentkezés
A felhasználó meg tudja változtatni belépéshez szükséges jelszavát.	Adatmódosítás, Adatbázis kezelés
A felhasználó meg tudja változtatni belépéshez szükséges emailcímét.	Adatmódosítás, Adatbázis kezelés
A felhasználó meg tudja nézni a játék előzményeit.	Profil megnézés
A felhasználó meg tudja nézni a mindeddig elért pontjait	Profil megnézés
A felhasználó el tudja indítani a játék folyamatot.	Játék, Esemény kezelés
A felhasználó tud passzolni.	Passz, Esemény kezelés
Ha nem érkezik tét, 6 darab egymást követő passz után a játék automatikusan véget ér.	Licitálás, Passz, Esemény kezelés
A felhasználó tud adott mennyiségű tétet tenni.	Licitálás, Esemény kezelés

A felhasználó csak a magasabb értékű tétet választhat.	Licitálás, Esemény kezelés
Két egymást követő passz után a tét rakás véget ér.	Licitálás, Passz, Esemény kezelés
Aki utoljára tett tétet az lesz a bemondó.	Játék, Esemény kezelés
Akik passzoltak lesznek a támadók.	Esemény kezelés
Ha a felhasználó a játék bemondója meg tudja nézni a talont.	Talon csere, Esemény kezelés
Ha a felhasználó a játék bemondója, ki tudja cserélni a kezében lévő kártyáit a talonban lévők kell.	Talon csere, Esemény kezelés
Ha a felhasználó a játék bemondója kihagyhatja a talon megnézését.	Talon csere, Esemény kezelés
A felhasználó a nála lévő kártyákból választhat.	Lépés
A játék 10 körből áll.	Játék
Minden körben 3 kártya kerül az asztalra, egy darab minden játékostól.	Lépés, Esemény kezelés
A kör végén kiértékelődik kinyerte az adott kört.	Játék, Esemény kezelés
Az előző kör nyertes kezdi a következőt.	Játék, Esemény kezelés
A bemondó nyeri a játékot, ha legalább 6 kört nyert.	Játék, Esemény kezelés
A védők nyernek, ha közösen legalább 5 kört nyernek.	Játék, Esemény kezelés
A játék végén a felhasználók választhatnak, hogy kilépnek vagy új játékot kezdenek a már meglévő játékosokkal.	Kilépés, Új játék, Esemény kezelés
Ha kilépnek minden ki számára véget ér a játék és mindenki visszatér a játékközpontba.	Kilépés, Esemény kezelés
Ha minden játékos az új játékot választotta, a tét tételtől új játék indul.	Új játék, Esemény kezelés
A játék során bármikor ki tudnak lépni a játékosok, ezzel mindenki számára véget vetve a játéknak.	Kilépés
A regisztrált felhasználók és adataik el vannak mentve egy adatbázisba.	Adatbázis kezelés
Belépéskor a felhasználó kap egy token-t.	Bejelentkezés, Authentikáció
Belépni csak egy már regisztált felhasználó tud.	Bejelentkezés, Authentikáció
A jelszónak minimum 5 karakteresnek kell lennie.	Regisztráció, Adatmódosítás, Authentikáció
Az email címnek meg kell felelni az email formátumnak.	Regisztráció, Adatmódosítás, Authentikáció
Kijelentkezéskor ez a token törlődik.	Kijelentkezés
Egy fiókba egyszere csak egy felhasználó lehet bejelentkezve.	Bejelentkezés, Authentikáció

A játék végén a nyert vagy veszített tét hozzáadódik, vagy kivonódik a felhasználók eddig elért pontjaihoz.	Játék, Esemény kezelés
A játék végén az adatok az adott játékról lementődnek egy adatbázisba.	Esemény kezelés, Adatbázis kezelés
Ha a felhasználó változtaja a jelszavát/emailcímét az adatbázisban is változik ez az információ.	Adatmódosítás, Adatbázis kezelés

2.3 Rendszerterv



2. ábra Rendszerterv

A backend és a frontend közötti kommunikáció REST alapú API-n keresztül valósul meg. A REST architektúra lehetővé teszi az erőforrások egységes és jól strukturált elérését HTTP protokollon keresztül, amely jól illeszkedik a Vue-alapú Single-Page Application működéséhez. Az API réteg felel a játéklogika műveleteinek végrehajtásáért, valamint a kérések és válaszok megfelelő kezeléséért.

Az alkalmazás adatainak tartós tárolása relációs adatbázis-kezelő rendszerek segítségével történik. A backend úgy kerül kialakításra, hogy több adatbázis-kezelő rendszerrel is kompatibilis legyen, különösen MySQL és PostgreSQL esetén. Ezen adatbázisokat csak a backend éri el.

2.4 Tervezett technológiák

A weboldal Single-Page Application (SPA)[4] architektúrában készül, amelynek frontend része a Vue, JavaScript keretrendszer felhasználásával kerül megvalósításra. A SPA megközelítés lehetővé teszi a gyorsabb és folyamatosabb felhasználói élményt, mivel az oldal újratöltése nélkül képes dinamikusan frissíteni a tartalmat. A Vue -t pedig kifejezetten SPA fejlesztésre optimalizált eszköztára jellemzi.

A fejlesztési folyamat során a Vite eszközkészlet kerül alkalmazásra, amely gyors build-elést és hatékony fejlesztői környezetet biztosít. A Vite fejlesztői szervere és Hot Module Replacement (HMR) funkciója jelentősen felgyorsítja a fejlesztést, mivel a kód módosításai azonnal megjelennek az alkalmazásban, teljes újratöltés nélkül.

A backend komponens Golang programozási nyelven fog készülni. A Go nyelv választását elsősorban a jó teljesítménye, hatékony párhuzamossági modellje és egyszerű, jól karbantartható szintaxisa indokolja. Ezek a tulajdonságok különösen alkalmassá teszik valós idejű kommunikációt és több egyidejű felhasználót kezelő webalkalmazások backend fejlesztésére.

Az alkalmazás két különböző adatbázis-kezelő rendszerrel is kompatibilis lesz: MySQL-lel és PostgreSQL-lel. Ennek célja a rendszer rugalmasságának növelése. Ennek célja a rendszer rugalmasságának növelése, valamint annak bemutatása, hogy az adatkezelési réteg megfelelő absztrakcióval leválasztható a konkrét adatbázis-implementációról

A weboldal a Docker konténerizációs technológia használatára előkészített módon kerül kialakításra. A projekt részeként Dockerfile-ok készülnek az egyes komponensekhez, amelyek lehetővé teszik az alkalmazás konténerben történő futtatását. A konténerizáció megkönnyíti az alkalmazás telepítését, csökkenti a környezetfüggő hibák esélyét, valamint jól illeszkedik a modern, cloud-native szemlélethez. A fejlesztés során az alkalmazás működése Docker környezetben is tesztelésre kerül, ezáltal biztosítva a reprodukálható futtatási környezetet és a későbbi egyszerű telepíthetőséget.

3 Technológiák

Az alábbi fejezetben ismertetem azokat a technológiákat, valamint a tervezési és fejlesztési elveket, amelyekre a weboldal megvalósítása során támaszkodom.

3.1 Vue SPA + Vite

A hagyományos webalkalmazások frontend és backend logikájának nagy rész a szerveroldalon fut. A felhasználó által indított kérések (például oldalváltás vagy űrlapbeküldés) hatására a szerver feldolgozza a kérést, majd egy teljes HTML oldallal válaszol. Minden navigációs lépés új HTTP kérést és a teljes oldal újbóli betöltését eredményezi.

Ezzel szemben SPA (Single-Page Application) architektúra lényege, hogy az alkalmazás egyetlen HTML oldalt tölt be, majd a felhasználók interakcióinak hatására a tartalom dinamikusan frissül, az oldal újratöltése nélkül. SPA architektúrában a frontend és a backend jól elkülönül egymástól, és jellemzően API-n keresztül kommunikálnak. Ez gyors felhasználói élményt biztosít. Az elkülönített frontend–backend architektúra és az API-alapú kommunikáció lehetővé teszi az alkalmazás későbbi bővítését és skálázását anélkül, hogy a teljes rendszert újra kellene tervezni. Az SPA architektúra jól illeszkedik a valós idejű kommunikációhoz

A Vue.js[8] egy modern JavaScript keretrendszer, amely kifejezetten alkalmas SPA típusú webalkalmazások fejlesztésére. A Vue.js komponensalapú felépítése lehetővé teszi a felhasználói felület logikus egységekre bontását, ami jól illeszkedik az SPA működési modelljéhez. A keretrendszer beépített reaktív adatkezelése biztosítja, hogy az adatok változására a felhasználói felület automatikusan frissüljön, ezáltal javítva a felhasználói élményt.

A navigálást az SPA komponenseink/oldalaink között a Vue Router[6] intézi. A Vue Router a Vue hivatalos kliensoldali útvonalkezelési megoldása. A böngésző URL-jét összekapcsolja a felhasználó által megjelenített tartalommal. A navigáció során az URL ennek megfelelően frissül.

A fejlesztés során a Vite[7] eszközkészlet használatára is sor kerül, mivel hatékonyan támogatja a Vue-alapú Single-Page Application (SPA) fejlesztést. A Vite egy

modern frontend fejlesztői eszközkészlet, amely jelentősen hozzájárul a gyorsabb és hatékonyabb fejlesztési folyamathoz.

A Vite beépített fejlesztőszervert biztosít, amely lehetővé teszi az alkalmazás azonnali futtatását, kipróbálását és tesztelését fejlesztés közben. Az úgynevezett Hot Module Replacement (HMR) funkció lehetővé teszi, hogy a már futó webalkalmazás egyes részei frissüljenek anélkül, hogy a teljes oldal újratöltésére lenne szükség, ezáltal felgyorsítva a fejlesztési ciklust.

A fejlesztési mód mellett a Vite éles környezetre optimalizált buildelési folyamatot is biztosít. Ennek során az alkalmazás JavaScript, CSS és egyéb statikus erőforrásai hatékonyan csomagolásra és optimalizálásra kerülnek, ami gyorsabb betöltési időt és jobb teljesítményt eredményez.

3.2 Go Programing Language (Golang, Go)

A backend fejlesztés során a Golang[9] programozási nyelv kerül alkalmazásra. A Go egy modern, statikusan típusos, fordított programozási nyelv. Beépített párhuzamosság támogatással rendelkezik goroutine-ok és csatornák (channels) formájában. Ez lehetővé teszi az alkalmazás számára több feladat egyidejű, hatékony kezelését. Különösen előnyös hálózati kommunikációt megvalósító backendnél.

Egy online kártyajáték esetében különösen fontos a gyors válaszidő és a több egyidejű felhasználó kezelése, amelyet a Go párhuzamosságot megoldó modelje támogat. Emellett a nyelv egyszerű felépítése és jól karbantarthatósága hozzájárul ahhoz, hogy a backend kód hosszú távon is könnyen bővíthető és továbbfejleszthető legyen.

A Go rendelkezik beépített HTTP csomagokkal, amelyek hatékonyan lehetővé teszik a REST alapú API végpontok megvalósítását. Ezeken keresztül a kliensoldali alkalmazás strukturált formában, jellemzően JSON adatok segítségével kommunikál a szerverrel.

Az alkalmazás adatainak tartós tárolása relációs adatbázis-kezelő rendszerek segítségével történik. A Go adatbázis-kezelő csomagjai lehetővé teszik a hatékony és biztonságos adatbázis-műveletek végrehajtását.

3.3 Cloud native patterns

A cloud-native patterns olyan tervezési és fejlesztési irányelvek összességét jelentik, amelyek egy olyan alkalmazás kialakítását segítik elő, amely hatékonyan képes alkalmazkodni a változó terheléshez, valamint stabilan és rugalmasan üzemeltethető felhőalapú környezetben.

A cloud-native szemlélet öt alapvető tulajdonságot határoz meg[5], amelyek elérésére törekedhetünk. Ezeken belül számos különböző megoldás és módszer létezik, amelyek közül a fejlesztés során megválasztható, hogy melyek kerülnek alkalmazásra. A cél ideális esetben az, hogy minél több ilyen irányelv érvényesüljön, ugyanakkor fontos szem előtt tartani az alkalmazás jellegét, összetettségét és valós igényeit, annak érdekében, hogy csak olyan megoldások kerüljenek tervezésre, amelyek tényleges hozzáadott értéket képviselnek.

3.3.1 Skálázhatóság

A skálázhatóság azt fejezi ki, hogy egy rendszer milyen mértékben képes alkalmazkodni a növekvő terheléshez anélkül, hogy a teljesítmény vagy a megbízhatóság jelentősen romlana. A skálázás alapvetően két fő irányba történhet: vertikális és horizontális skálázás formájában.[10]

A vertikális skálázás során egy meglévő rendszer erőforrásait növeljük, például több processzormagot, nagyobb memóriát vagy gyorsabb háttértárat biztosítunk számára. Ebben az esetben az alkalmazás egyetlen példányban fut, azonban nagyobb számítási kapacitással rendelkezik.

A horizontális skálázás során az alkalmazás több példányban fut párhuzamosan, és a terhelés ezek között kerül elosztásra. Ez a megközelítés különösen jól illeszkedik a cloud-native szemlélethez, mivel lehetővé teszi a rugalmas bővíthetőséget és a magas rendelkezésre állást. A megvalósításához többféle architekturális és tervezési megoldás létezik. A stateless felépítés és a cache alkalmazása csupán példák azon technikák közül, amelyek hozzájárulhatnak a skálázható működéshez.

A jelenlegi webalakalmazásnál, az szeretnénk, hogy az alkalmazás architektúrája lehetőséget biztosítson a későbbi skálázásra, elsősorban horizontális irányban. [2]

3.3.2 Lazán csatolt felépítés (Loose Coupling)

A lazán csatolt (loosely coupled) komponensek minimális ismerettel rendelkeznek egymás belső működéséről. Ezek a komponensek viszonylag függetlenek, és jellemzően stabil, jól definiált absztrakciókon keresztül kommunikálnak egymással. A komponensek szabadabban frissíthetők, újratelepíthetők, vagy akár teljesen újra is írhatók anélkül, hogy ez jelentős hatással lenne a többi komponensre.

Ennek megfelelően a rendszer tervezése során cél, hogy a komponensek világosan elkülönüljenek egymástól: a frontend és a backend egymással REST API-n keresztül kommunikál, míg az adatkezelés külön adatbázisrétegben valósul meg. A konfiguráció külön kezelése a kódtól csökkenti az egyes komponensek egymásra utaltságát. Ez a felépítés lehetővé teszi az egyes részek önálló fejlesztését, cseréjét vagy továbbfejlesztését, miközben a rendszer egésze stabil marad.

3.3.3 Ellenállóképesség

Az ellenállóképesség egy rendszer azon képességét jelenti, hogy hibák vagy váratlan körülmények esetén is működőképes maradjon, illetve képes legyen gyorsan helyreállni. A tervezés és a megvalósítás során cél, hogy előre azonosítsuk azokat a lehetséges hibaforrásokat, amelyek problémát okozhatnak a rendszer működésében.

Ennek érdekében törekedni kell a hibák izolálására, valamint egészségellenőrző (health check) végpontok és állapotjelzések alkalmazására.

Fontos szempont továbbá az időtúllépések kezelése és a megfelelő hibakezelési mechanizmusok kialakítása, annak érdekében, hogy egyetlen komponens meghibásodása ne vezessen a teljes rendszer leállásához.

A terhelés megfelelő elosztása szintén hozzájárul ahhoz, hogy elkerülhető legyen az egyes komponensek túlterhelése.

A gyors, adatvesztés nélküli újraindíthatóság biztosítása szintén az ellenállóképesség fontos eleme. Ezek az elvek együttesen járulnak hozzá ahhoz, hogy a rendszer részleges meghibásodás esetén is stabilan és megbízhatóan működjön.

3.3.4 Menedzselhetőség

A menedzselhetőség azt fejezi ki, hogy egy rendszer működése mennyire könnyen módosítható külső beavatkozásokkal, jellemzően a forráskód megváltoztatása nélkül.

A menedzselhetőséget jelentősen növeli a deklaratív konfigurációk alkalmazása, amelyek egyértelműen leírják az alkalmazás futtatásához szükséges erőforrásokat és beállításokat. Emellett fontos szempont a szabályos indítás és leállítás kezelése, amely lehetővé teszi a karbantartási műveletek és frissítések biztonságos végrehajtását anélkül, hogy az alkalmazás működése sérülne.

A konténerizációs megoldások, például a Dockerfile és a Docker Compose konfigurációk használata, hozzájárulnak az alkalmazás egyszerű telepítéséhez és egységes üzemeltetéséhez különböző környezetekben.

3.3.5 Megfigyelhetőség

A megfigyelhetőség azt fejezi ki, hogy egy rendszer belső állapota és működése mennyire jól követhető és értelmezhető a külső visszajelzések alapján. Célja, hogy az alkalmazás viselkedése átlátható legyen, és a felmerülő hibák, teljesítményproblémák vagy rendellenességek gyorsan azonosíthatók legyenek anélkül, hogy bele kéne nyúlnunk a rendszer belső működésébe.

A megfigyelhetőség a naplózásra, a metrikák gyűjtésére, valamint az állapotjelzések és nyomkövetés alkalmazására épül. A naplózás lehetővé teszi az események és hibák visszakövetését, míg a metrikák segítségével a rendszer teljesítménye és erőforrás-használata figyelhető meg. Az állapotjelzések és egészségellenőrzések pedig információt adnak arról, hogy az alkalmazás aktuálisan képes-e a kérések kiszolgálására.

3.3.6 Konténerizáció

Az egyik alapvető technológiai eszköz, ami ebben tud nekünk segíteni az a konténerizáció. Nem megvalósítja ezeket a funkciókat/elveket, hanem egy alapot biztosit azok eléréséhez. Segíti a skálázhatóságot azáltal, hogy az alkalmazás gyorsan és konzisztensen indítható több példányban, támogatja a lazán csatolt felépítést az egységes interfészek révén, valamint hozzájárul a menedzselhetőséghez és a megfigyelhetőséghez is az egységes futtatási modellnek köszönhetően.

3.4 Valós Idejű Kommunikáció

A valós idejű kommunikáció olyan, amely lehetővé teszi az adatok azonnali továbbítását a kliens és a szerver között, minimális késleltetéssel. Ennek köszönhetően a felhasználók az webalkalmazás állapot változásait késedelem nélkül érzékelhetik, ami különösen fontos interaktív webalkalmazások esetén.

Egy online kártyajáték működése során elengedhetetlen, hogy a játékosok azonnal értesüljenek a játék változásairól (például a kijátszott lapokról vagy a kör végéről). A hagyományos kérés–válasz alapú kommunikáció (pl. REST API) önmagában nem alkalmas erre, mivel a kliensnek folyamatosan lekérdezéseket kellene indítania a szerver felé, ami késleltetést és felesleges hálózati terhelést eredményezne.

A webes környezetben több különböző megoldás is létezik valós idejű kommunikáció megvalósítására.

Egy modernebb megoldás a **Server-Sent Events** (SSE)**Hiba! A hivatkozási forrás nem található.**, amely lehetővé teszi, hogy a szerver egyirányú kapcsolatban küldjön adatokat a kliens felé. Bár ez bizonyos esetekben hatékony, nem támogatja a kétirányú kommunikációt, amely egy online többjátékos játék esetében alapvető követelmény.

A **WebRTC** (Web Real-Time Communication)**Hiba! A hivatkozási forrás nem található.** egy olyan technológia, amely közvetlen, peer-to-peer kapcsolatot tesz lehetővé a kliensek között. Elsődlegesen valós idejű multimédiás kommunikációra (hang, videó, adatcsatornák) lett tervezve, és képes alacsony késleltetésű adatátvitelre a böngészők között. A WebRTC előnye, hogy bizonyos esetekben a kommunikáció közvetlenül a kliensek között zajlik, ami csökkentheti a szerver terhelését.

Ugyanakkor a WebRTC használata összetett infrastruktúrát igényel. Emellett a WebRTC elsődlegesen kliens-kliens kommunikációra optimalizált, ami kevésbé illik olyan alkalmazásokhoz, ahol a szerver központi szerepet játszik.

A **WebSocket** egy olyan kommunikációs protokoll, amely tartós, kétirányú kapcsolatot biztosít a kliens és a szerver között. A kapcsolat létrejötte után mindkét fél képes adatokat küldeni a másiknak anélkül, hogy új HTTP kapcsolatot kellene létrehozni. Ezzel csökkenti a hálózati terhelést is, mivel nem lesz szükség folyamatos HTTP kérések küldésére. Ez alacsony késleltetést és hatékony adatcserét tesz lehetővé. További előnye,

hogy széles körben támogatott a modern böngészőkben, valamint jól integrálható a Go alapú backend környezetbe.

Az általunk implementálni készült webalkalmazásban a WebSocket megközelítése (kliens-szerver kommunikáció) jobban illik az állatunk elérni kívánt célokhoz.

4 Implementáció

4.1 Webalkalmazás felépítése

A tervezés fejezetben bemutatott tervek alapján a következő kép lett implementálva.

4.1.1 Frontend

A frontend egy SPA, ami a következő módon lett megvalósítva:

App.vue(részlet):

```
<template>
  <div class="app-wrapper">
    <the-header></the-header>
    <main class="main-content">
      <router-view v-slot="slotProps">
        <keep-alive :include="['Profile']">
          <component :is="slotProps.Component"></component>
        </keep-alive>
      </router-view>
    </main>
    <the-footer></the-footer>
  </div>
</template>
```

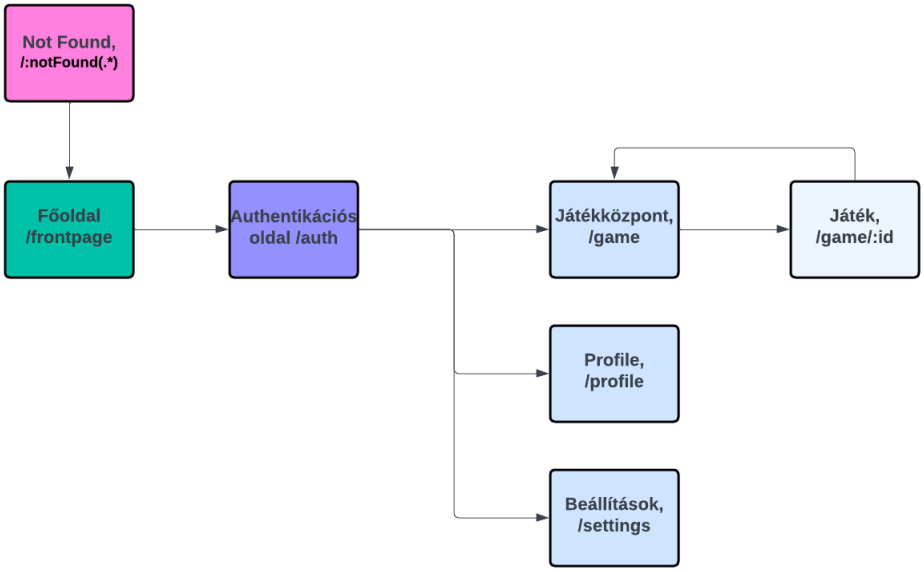
Az App.vue fájl az alkalmazás fő elrendezését valósítja meg. Tartalmazza azokat az elemeket, amelyek az alkalmazás minden oldalán állandóan megjelennek, mint például a fejléc (header) és a lábléc (footer), valamint azt a tartalmi részt is, amely dinamikusan változik a navigáció során.

router.js(részlet):

```
const router = createRouter({
    history: createWebHistory(),
    routes: [
        {path: '/', redirect: '/frontpage'},
        {path: '/frontpage', component: FrontPage},
        {path: '/auth', component: AuthPage},
        {path: '/game', component: GameHub,},
        {path: '/game/:id', component: GamePage},
        {path: '/profile', component: Profile},
        {path: '/settings', component: Settings},
        {path: '/:notFound(.*)', component: NotFound}
    ]
});
```

A dinamikus tartalom megjelenítéséért a Vue Router által biztosított router-view komponens felelős. Ez a komponens az aktuális útvonalhoz tartozó nézetet jeleníti meg. Az, hogy mely útvonalhoz mely komponens tartozik, egy különálló **router.js** fájlban kerül definiálásra, amely az alkalmazás navigációs logikáját tartalmazza. A router-view ezen konfiguráció alapján tölti be dinamikusan a megfelelő komponenst.

A keep-alive komponens alkalmazásával lehetőség van bizonyos nézetek állapotának megőrzésére navigáció során. Ennek köszönhetően az érintett komponensek nem kerülnek újrarenderelésre minden útvonalváltáskor, ami javítja a felhasználói élményt, valamint csökkenti az újrarenderelésből adódó teljesítményköltségeket. Azért a Profile komponens az egyetlen, amit szeretnénk betöltve tartani, mert amíg a felhasználó be van jelentkezve addig az információ azon az oldalon nem nagyon fog változni.



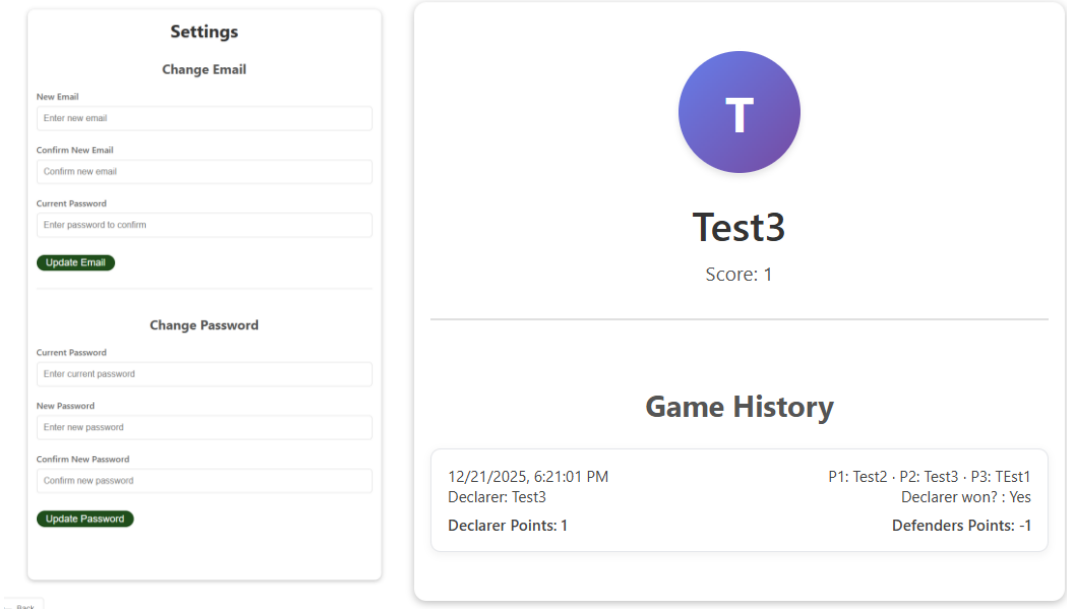
3. ábra Frontend Routing

A webalkalmazás főoldala (/frontpage) két különböző állapottal rendelkezik, amelyek meghatározzák a felhasználó számára elérhető funkciókat: a kijelentkezett és a bejelentkezett állapot. A játékfunkciók eléréséhez a felhasználónak fiókot kell létrehoznia, majd be kell jelentkeznie az oldalra. A kijelentkezett állapotban a főoldalon elérhető „Log In” gomb az authentikációs oldalra (/auth) navigálja a felhasználót.

Az authentikációs oldalon a felhasználó egyszerűen válthat a bejelentkezés és a regisztráció funkciók között. Mindkét művelet űrlap (form) segítségével kerül megvalósításra.

A regisztráció során a felhasználónak meg kell adnia a nevét, e-mail címét és jelszavát. Az alkalmazás csak akkor fogadja el az e-mail címet, ha az megfelel az e-mail formátumra vonatkozó validációs szabályoknak, a jelszó pedig csak akkor érvényes, ha legalább öt karakter hosszúságú. A bejelentkezéshez ezzel szemben kizárólag az e-mail cím és a jelszó megadása szükséges.

A bejelentkezett felhasználó számára a főoldal három menüpontot kínál fel: GameHub(/game), Profile (/profile), Settings (/settings).



4. ábra Setting és Profile oldalak

A Profile oldalon a felhasználóval kapcsolatos információk kerülnek megjelenítésre, például az aktuális pontszám és a korábbi játékok adatai.

A Settings oldalon lehetősége van a felhasználónak emailt és jelszót módosítania.

A GameHub oldalra történő navigáláskor a komponens betöltésekor (mounted() lifecycle hook) elindul a WebSocket kapcsolat felépítése. Ezen az oldalon a felhasználó lehetőséget kap új játék kezdeményezésére.

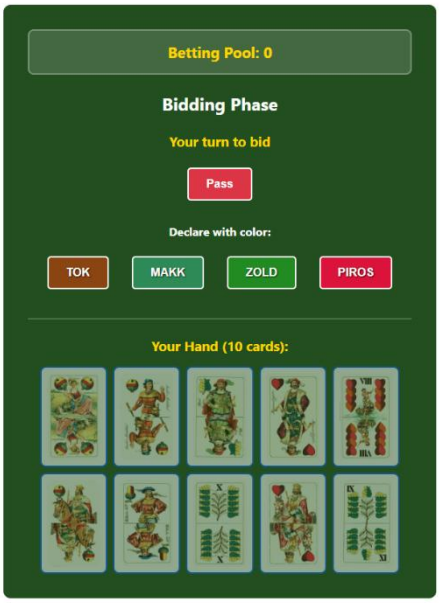
Játék indítása esetén az alkalmazás a /game/{gameID} útvonalra navigálja a felhasználót. A gameID egy generált azonosító, amely egy adott játékot reprezentál, és amelyhez pontosan három játékos csatlakozhat. A /game/{gameID} oldal megjelenítéséért a GamePage komponens felelős, amely a játék különböző fázisait elkülönített alkomponensekben kezeli.

GamePage.vue(részlet):

```
<template>
    <waiting-room v-if="!gameStarted"></waiting-room>
    <interruption-message :show="showInterruption" />
    <template v-if="gameStarted">
      <button class="exit-button" @click="showExitConfirmation">×</button>
       <exit-dialog
           :show="showExitDialog"
           @confirm="confirmExit"
           @cancel="showExitDialog = false"
       />
       <bidding-phase v-if="gamePhase === 'bidding'" />
       <talon-exchange v-if="gamePhase === 'talon_exchange'" />
       <playing-area v-if="gamePhase === 'playing'" />
       <game-result v-if="gamePhase === 'ended'" />
    </template>
</template>
```

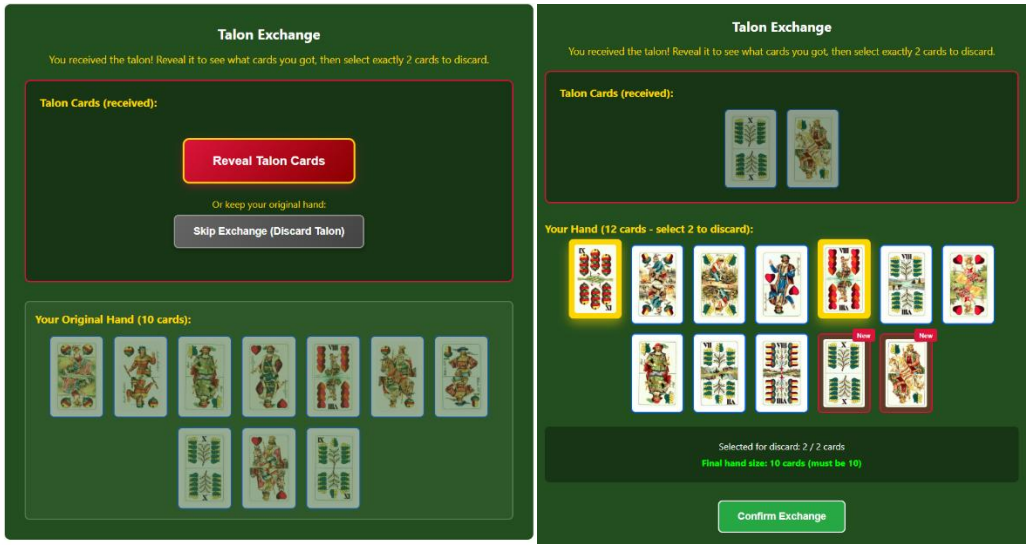
A waiting-room komponens akkor jelenik meg, amikor még nem csatlakozott elegendő játékos a játékhoz. Ezen a felületen a várakozó játékosok visszajelzést kapnak arról, hogy hány további játékosra van még szükség a játék elindításához.

A bidding-phase komponens a licitálás során aktív, ahol a játékosok a játékszabályoknak megfelelően licitálhatnak



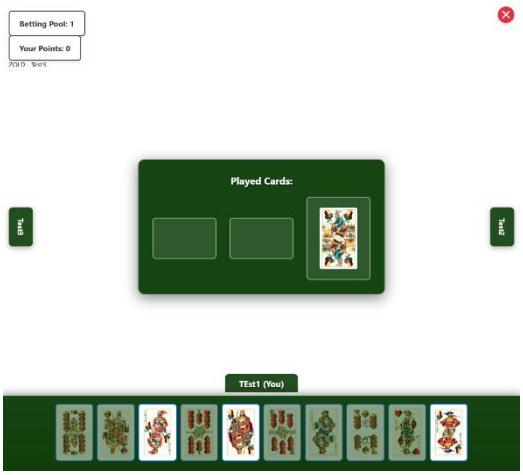
5. ábra Licitálás fázis kinézete

A talon-exchange komponens kizárólag a bemondó játékos számára interaktív. A többi játékos ebben a fázisban várakozó üzenetet lát, amíg a bemondó meghozza döntését.



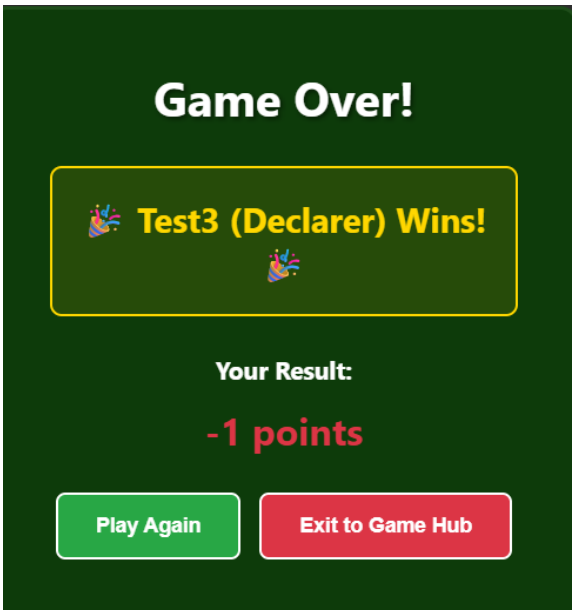
6. ábra Talon csere

A playing-area komponensben a játékosok látják az asztalt, a saját kártyáikat, valamint a játék aktuális állapotát. Amikor egy játékos sorra kerül, kattintással helyezheti le a kártyáját az asztalra.



7. ábra Játék kinézete

A game-result komponens akkor jelenik meg, amikor az utolsó játékos is lehelyezte a kártyáját, és a játék véget ér. Ekkor az alkalmazás lehetőséget kínál egy új játék indítására.



8. ábra Nyertes hírdető, játék vége kiírás

A játék során végig elérhető egy **Exit** gomb, amely lehetőséget biztosít a játék elhagyására. A gombra kattintva egy megerősítő párbeszédablak jelenik meg, amely rákérdez a kilépési szándékra. Ha valaki elhagyja a játékot, a játék véget ér mindenki számára.

Az alkalmazás tartalmaz egy külön **NotFound** komponenst is, amely akkor jelenik meg, ha a felhasználó olyan útvonalra navigál, amely nem létezik. Ez biztosítja a felhasználó számára az egyértelmű visszajelzést hibás URL esetén.

A megvalósítás során hasznát vettem az úgy nevezet store-oknak. A store egy központi állapotkezelő egység, amely az alkalmazás különböző komponensei által megosztott adatokat kezeli. A frontend alkalmazásban két külön store került kialakításra, amelyek eltérő felelősségi köröket látnak el.

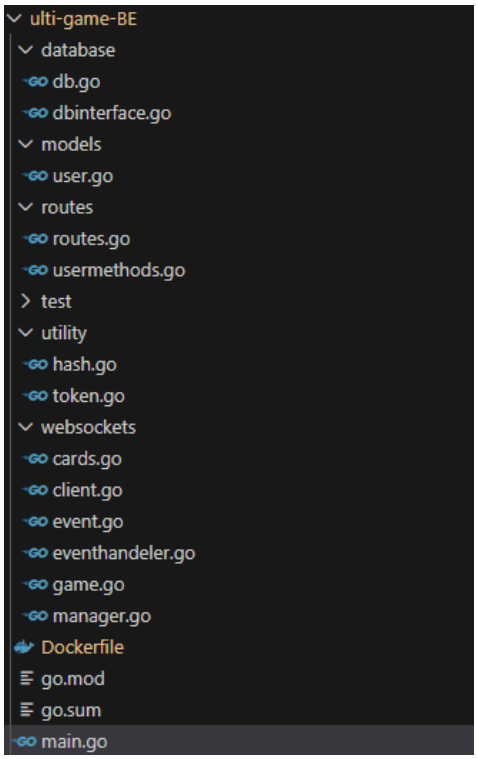
Az auth.js store a felhasználóval kapcsolatos adatok kezeléséért felel. Itt kerül eltárolásra a bejelentkezés során a backend által visszaadott hitelesítési token, amely szükséges a felhasználói adatokkal kapcsolatos, jogosultságot igénylő kérések elküldéséhez. Emellett a store tartalmazza a belépett felhasználó alapvető adatait, például az e-mail címét, valamint az aktuális pontszámát.

A ws.js store a WebSocket kapcsolattal és a játék aktuális állapotával kapcsolatos adatokat kezeli. Itt kerül eltárolásra maga a WebSocket kapcsolat, valamint minden olyan információ, amely a valós idejű játék működéséhez szükséges. lyen adatok például a játék azonosítója (gameID), amely alapján az alkalmazás a megfelelő játékoldalra navigál, a

játékos aktuális kártyái, valamint a többi játékos neve, akikkel az adott játékban részt vesz.

A WebSocket kommunikáció átláthatóbb kezeléséhez egy külön Event osztály került kialakításra. Ez az osztály kezeli a WebSocketen küldött és fogadott üzeneteket, valamint megkönnyíti, azok értelmezését és irányítását az alkalmazáson belül.

4.1.2 Backend



9. ábra A fájl rendszere a backendnek

A backend alkalmazás belépési pontja a main.go fájl, amelyből az alkalmazás teljes működése indul. Ebben a fájlban történik az adatbázis-kapcsolat inicializálása (a db.go modul segítségével), a HTTP szerver létrehozása és indítása a Gin keretrendszer alkalmazásával, valamint az útvonalak regisztrálása a route.js segítségével.

A backend szerver kezelését a Go nyelvhez készült Gin**Hiba! A hivatkozási forrás nem található.** webes keretrendszer segítségével valósítottam meg. A Gin biztosított egy HTTP routert biztosít, alkalmas REST API-k és WebSocket-alapú kommunikáció kezelésére. *Az adatbázis-kezelés részletes bemutatása az adatbázissal foglalkozó fejezetben történik.*

A backend egyik központi eleme a routes.go fájl, amely az alkalmazásba érkező HTTP kérések útvonalainak (route-ok) regisztrálásáért és azok megfelelő függvényekhez való irányításáért felel.

routes.go(részlet):

```
func AvaibleRoutes(server *gin.Engine) {

    manager := websockets.NewManager()

    server.POST("/auth", authenticationMethod)
    server.POST("/logout", logout)
    server.GET("/profile", getProfile)
    server.PUT("/change-email", changeEmail)
    server.PUT("/change-password", changePassword)
    server.GET("/games", getGames)
    .
    .
    .
}
```

<u>POST végpontok</u>

A /auth végpont a felhasználók regisztrációját és bejelentkezését kezeli, míg a /logout végpont a kijelentkezési folyamatért felel.

<u>GET végpontok</u>

A /profile végpont a bejelentkezett felhasználó profiljához tartozó adatok lekérésére szolgál, a /games pedig a lejátszott játékok adatainak lekérdezését teszi lehetővé.

<u>PUT végpontok</u>

A /change-email és /change-password végpontok a felhasználói beállítások módosításához kapcsolódó kéréseket kezelik, például az e-mail cím és a jelszó megváltoztatását.

A //websocket megjegyzéssel jelölt rész felelős a WebSocket kapcsolat inicializálásáért. A /game végpontra érkező kérés esetén a WSHandler függvény kerül meghívásra, amely létrehozza és kezeli a WebSocket kapcsolatot. A WebSocketkel kapcsolatos logika külön package-ben került elhelyezésre. A játék működéséhez és eseményeihez kapcsolódó elemek szintén ebben a package-ben vannak.

A manager.go fájl a WebSocket kapcsolatok központi kezeléséért felelős, például az aktív kapcsolatok nyilvántartásáért és koordinálásáért. A client.go fájl ezzel szemben

egyetlen klienshez tartozó WebSocket kapcsolat adatait és állapotát kezeli, elkülönítve az egyes kliensekhez tartozó információkat.

A felhasználók tárolására létrehoztam egy user.go fájlt amiben található a user struktúra és az azzal kacsolatos műveletek. Például a Save() ami segít az újjonnan regisztrált felhasználók adatbázisba való beillesztésében segít.

A felhasználók adatainak kezelésére egy külön user.go fájl került létrehozásra, amely tartalmazza a felhasználói struktúrát és az ahhoz kapcsolódó műveleteket. Ilyen például a Save() metódus, amely az újonnan regisztrált felhasználók adatainak adatbázisba történő mentését végzi.

A usermethods fájl tartalmazza a routes.go-ból meghívott, felhasználóval kapcsolatos függvényeket, például az authenticationMethod, logout, changePassword és changeEmail függvényeket.

A utility csomag az ezekhez kapcsolódó segédfüggvéyneket foglalja magába, mint például a jelszavak biztonságos hashelése, a megadott jelszó összehasonlítása a tárolt hash értékkel, valamint a hitelesítési tokenek generálása és ellenőrzése.

4.1.3 Adatbázis

Jelenleg két különböző adatbázis-kezelő rendszert képes használni: a MySQL-t és a PostgreSQL-t. Az, hogy az alkalmazás melyik adatbázist alkalmazza, az indítás során kerül meghatározásra környezeti változók segítségével.

Ennek megvalósításához egy adatbázis interfészre van szükségünk. Ez az interfész környezeti változókból megkapja mely adatbázis-kezelőhöz tartozó megoldást használja.

A PostgreSQL és a MySQL között különbség van az SQL lekérdezésekben használt placeholderek kezelésében: míg a PostgreSQL számozott placeholdereket használ (például $1, $2), addig a MySQL kérdőjeles (?) jelölést alkalmaz. Annak érdekében, hogy az alkalmazás adatbázis-független módon tudja kezelni ezeket a különbségeket, minden adatbázis-művelet során egy NormalizeQuery nevű függvény kerül meghívásra. Ez a függvény a lekérdezést automatikusan a kiválasztott adatbázis-kezelő által elvárt szintaxisra alakítja át.

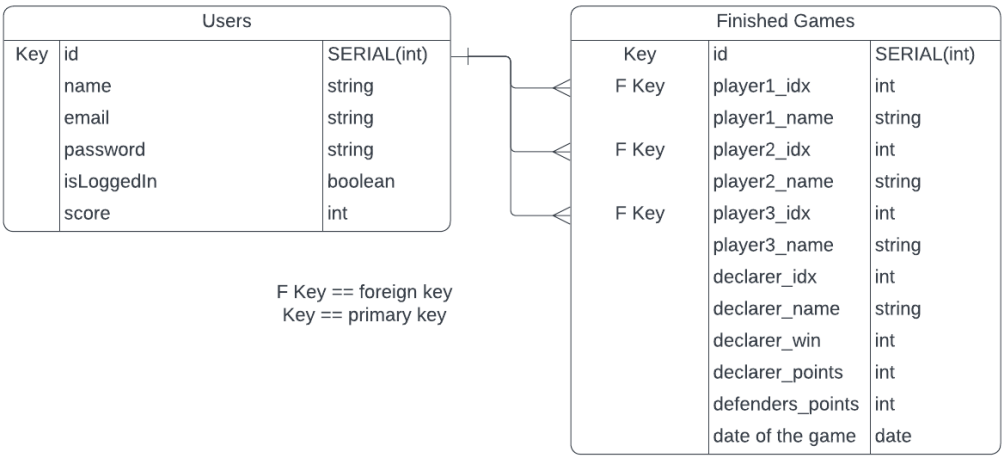
Ezen felül minden támogatott adatbázis-kezelőhöz külön, adatbázis-specifikus CreateTables függvény tartozik, amely az adott rendszer szintaxisának és sajátosságainak megfelelően hozza létre a szükséges adatbázis-struktúrákat.

Ez a megoldás lehetővé teszi, hogy az alkalmazás ugyanazzal a kódbázissal több adatbázis-kezelő rendszert is támogasson.

dbinterface.go (részlet):

```go
var pgPlaceholder = regexp.MustCompile(`\$[0-9]+`)

// converts Postgres ($n) into MySQL '?'
func NormalizeQuery(d DbInterface, q string) string {
    if d != MySQL {
        return q
    }
    // replace all $n with ?
    return pgPlaceholder.ReplaceAllString(q, "?")
}
```



10. ábra Adatbázis diagram

Az adatbázisban két táblánk van. Az egyik a regisztrált felhasználókat tárolja a másik a lejátszott játékokat.

A regisztrált felhasználók táblában az elsődleges kulcs az id. Ez egy SERIAL típusú kulcs, ami automatikusan növekvő, egyedi azonosítót rendel minden új rekordhoz, biztosítva ezzel az egyértelmű azonosíthatóságot. Ebben a táblában rögzítésre kerül a felhasználó neve, e-mail címe, a jelszó biztonságos, hashelt formája, valamint az eddig elért összesített pontszáma. Az isLoggedIn mező azt a célt szolgálja, hogy nyomon

követhető legyen, a felhasználói fiók aktuálisan be van-e jelentkezve. Ennek segítségével elkerülhető, hogy ugyanazt a fiókot egyszerre több felhasználó használja.

A tábla – a regisztrált felhasználók táblájához hasonlóan – egy SERIAL típusú id mezőt használ elsődleges kulcsként, amely biztosítja az egyes játékok egyértelmű azonosíthatóságát. A táblában rögzítésre kerülnek a játékban részt vevő játékosok nevei és azonosítói, valamint a bemondó játékos neve és azonosítója. Ezen felül tárolásra kerül az adott játék kimenetele, vagyis, hogy a bemondó sikerrel járt-e A rendszer emellett eltárolja az egyes játékosok által megszerzett pontszámokat is. A játék adatokat kiegészítjük egy időbélyeggel (timestamp), amely azt rögzíti, hogy az adott játék mikor került lejátszásra.

4.2 Lényegesebb funkciók

4.2.1 HTTPS

A HTTPS (HyperText Transfer Protocol Secure) a HTTP protokoll biztonságos változata, amely titkosított kommunikációt biztosít a kliens és a szerver között. TLS (Trasnport Layer Security) protokollt használ, hogy a hálózaton továbbított érzékeny adatok ne legyenek lehallgathatóak, módosíthatóak egy harmadik fél által.

A webalkalmazás fejlesztése során self-signed tanúsítvány került alkalmazásra. Ez a megoldás lehetővé teszi a HTTPS használatát fejlesztési és tesztelési környezetben anélkül, hogy külső hitelesítésszolgáltató (Certificate Authority) bevonására lenne szükség. Ehhez egy privát kulcs (.key) és egy tanúsítvány (.crt) fájl került generálásra.

A következő lépés az, hogy a backend main.go függvényébe beállítottam a HTTPS támogatást:

```
srv := &http.Server{
    Addr:    ":3000",
    Handler: server,
}

// Start server in a goroutine
go func() {
    if err := srv.ListenAndServeTLS("server.crt", "server.key"); err != nil
    && err != http.ErrServerClosed {
        log.Fatalf("listen: %s\n", err)
    }
}()
```

Ez a kódrészlet egy HTTPS-képes szervert indít a 3000-es porton.

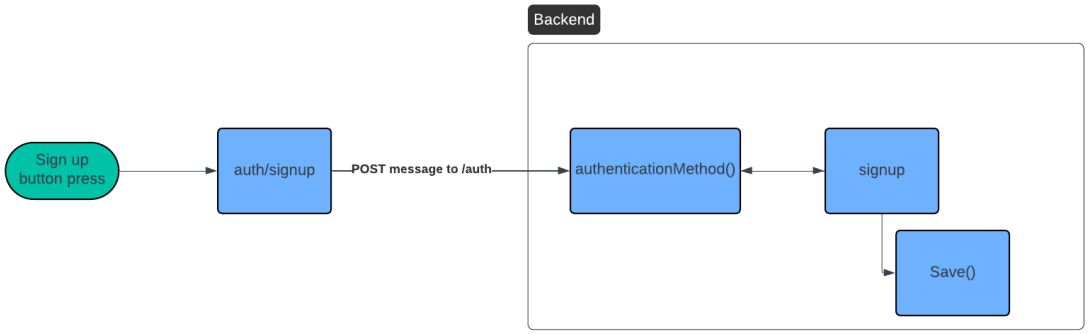
A vue oldalán arra kellett figyelni, kell hogy a Vite konfigurációs fájljában (vite.config.js) az API proxy beállításánál a cél URL-t „http”-ről át kellett írni „https”-re:

```
proxy: {
  '/api': {
    target: 'https://localhost:3000',
    changeOrigin: true,
    rewrite: path => path.replace(/^\/api/, '')
  }
},
```

Ezek mellett a docker-compose.yml fájlban gondoskodni kell arról, hogy az Nginx számára a megfelelő portok legyenek beállítva, és a HTTPS forgalom továbbítása megfelelően működjön.

4.2.2 Authentikáció:

Regisztráció



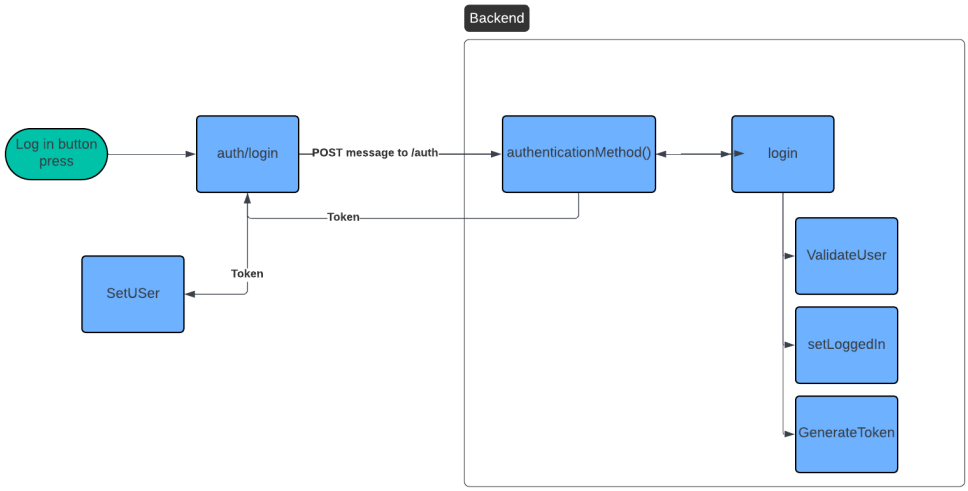
11. ábra Regisztráció folyamata (error handeling nélkül)

A frontend oldalon a felhasználói regisztráció a „Sign up” gomb megnyomásával indul, amely meghívja az auth/signup függvényt. Ez a függvény egy **POST** kérést küld a backend felé, amelynek törzsében (body) egy **JSON** objektum található. A JSON tartalmazza a felhasználó által megadott nevet, e-mail címet és jelszót, valamint egy state attribútumot, amely a "signup" értéket veszi fel. Ez az attribútum jelzi a backend számára, hogy regisztrációs műveletről van szó.

A backend oldalon a kérés fogadását a routes.go fájlban definiált útvonalkezelés végzi: server.POST("/auth", authenticationMethod). Ennek hatására a vezérlés a usermethods.go fájlban található authenticationMethod függvénybe kerül.

A authenticationMethod függvény a kérésben kapott state mező alapján megállapítja, hogy regisztrációs kérés érkezett. Ennek megfelelően meghívja az

ugyanebben a packageben/fáljban található signup függvényt. A signup függvény egy User objektumot vár paraméterként, majd a Save metódus segítségével megkísérli a felhasználó adatainak adatbázisba történő mentését. Amennyiben a felhasználó még nem létezik, a mentés sikeresen végrehajtódik; ellenkező esetben a rendszer hibát jelez (például „már létező felhasználó” esetén).



12. ábra Bejelentkezés folyamata (error handeling nélkül)

Bejelentkezés

A bejelentkezési folyamat a regisztrációhoz hasonló módon történik. A frontend oldalon a „Log in” gomb megnyomásával kerül meghívásra az auth/login függvény. Ez a függvény egy **POST** kérést küld a backend felé, amelynek törzsében egy **JSON** objektum található. A JSON az e-mail címet, a jelszót, valamint egy state attribútumot tartalmaz, amely ebben az esetben a "login" értéket veszi fel.

A backend oldalon a kérést továbbra is a routes.go fájlban definiált server.POST("/auth", authenticationMethod) végpont fogadja. A vezérlés a usermethods.go fájlban található authenticationMethod függvénybe kerül, amely a state mező értéke alapján ezúttal a login függvényt hívja meg.

A login függvény három fő lépést hajt végre. Elsőként a ValidateUser művelet ellenőrzi, hogy létezik-e a megadott e-mail címhez tartozó felhasználó, valamint hogy a megadott jelszó megegyezik-e az adatbázisban tárolt, hashelt jelszóval. Második lépésként a rendszer megvizsgálja, hogy a felhasználó jelenleg be van-e jelentkezve; amennyiben igen, a bejelentkezési kísérlet hibával megszakításra kerül. Ha a felhasználó még nincs bejelentkezve, az állapota ennek megfelelően frissítésre kerül.

Végül a rendszer a GenerateToken függvény segítségével hitelesítési tokent állít elő, amelyet válaszként visszaküld a frontend számára. A frontend oldalon a kapott token eltárolásra kerül, és a további, jogosultságot igénylő kérések során felhasználásra kerül.

4.2.3 WebSocketen keresztül küldött üzenet kezelés

Mind a frontend, mind a backend oldalon létrehozásra került egy **Event** struktúra (frontend oldalon osztályként, backend oldalon struktúraként). Az Event célja, hogy a WebSocketen keresztül küldött és fogadott üzenetek egységes formátumban legyenek kezelve.

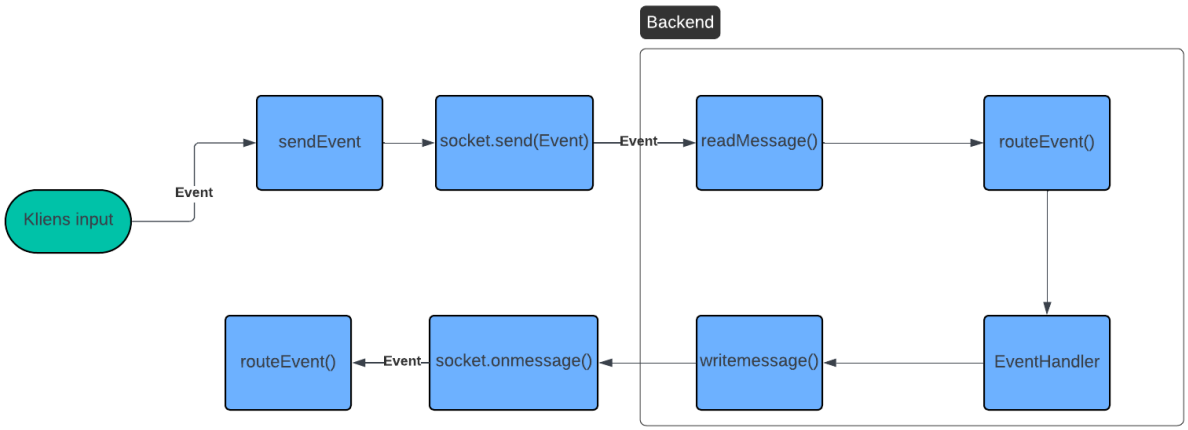
Az Event három fő részből áll. Egy **type** mezőből, amely az esemény típusát (nevét) határozza meg. Egy **id** mezőből, amely a játék azonosítóját (gameID) tartalmazza. Valamint egy **payload** mezőből, amely az eseményhez tartozó adatokat hordozza. A payload tartalma az esemény típusától függően változik.

A frontend oldalon a WebSocket kapcsolat létrehozása a GamePage komponens betöltésekor történik. A mounted életciklus-metóduson belül meghívásra kerül a ws/connect főfüggvény, amely WebSocket kapcsolatot kezdeményez a backend felé.

Sikeres kapcsolódás esetén a létrejött WebSocket objektum eltárolásra kerül socket néven. Ezt követően kerül regisztrálásra a socket.onmessage eseménykezelő, amely a backend felől érkező eseményeket és üzeneteket figyeli. Amennyiben a WebSocket kapcsolat lezárásra kerül, a socket.onclose eseménykezelő fut le.

A backend oldalon a WebSocket kommunikáció a WSHandler nevű függvény meghívásával indul el. Ez a függvény felelős a beérkező WebSocket kapcsolat fogadásáért és eltárolásáért.

A kapcsolat létrejötte után a szerver két külön goroutine-t indít el. A readMessages() goroutine folyamatosan figyeli, hogy érkezik-e üzenet a kliens felől, míg a writeMessages() goroutine a kliens felé küldendő üzenetek kezeléséért felel.



13. ábra WebSocket Event küldés

Esemény küldése:

A frontend oldalon többféle felhasználói művelet – például egy gomb megnyomása – válthat ki eseményt, amelyet az alkalmazás a backend felé továbbít. Ezek az események egységes formátumban, egy Event osztályba csomagolva kerülnek elküldésre. Az események elküldéséért a frontend oldalon az auth/sendEvent függvény felel, amely a WebSocket kapcsolat segítségével továbbítja az adatokat:

```
sendEvent(context, eventdata){
        const event = new Event(eventdata.type, eventdata.id,
        eventdata.payload);
      const wsocket = context.getters.socket;

      if (!wsocket || wsocket.readyState !== WebSocket.OPEN) {
          console.error('WebSocket is not connected. Cannot send event.');
           return;
      }

      try {
          wsocket.send(JSON.stringify(event));
      } catch (error) {
          console.error('Error sending WebSocket message:', error);
      }
  }
```

A backend oldalon a WebSocketen érkező üzenetek feldolgozását a client.go fájlban található readMessage függvény végzi. Ez a függvény beolvassa a beérkező üzenetet, majd az Event struktúrába történő unmarshal után kinyeri az eseményhez tartozó adatokat.

Ezt követően a manager.go fájlban található routeEvent függvény kerül meghívásra. Ez a függvény az esemény típusát (event.Type) használja fel annak meghatározására, hogy melyik eseménykezelő (handler) függvényt kell meghívni.

```go
type EventHandler func(event Event, c *Client) error

func (m *Manager) setupEventHandlers() {
    m.handlers[EventCardClicked] = CardClicked
    m.handlers[EventGameInit] = SendGameInit
    m.handlers[EventGameExit] = GameExit
    m.handlers[EventBidAction] = BidAction
    m.handlers[EventTalonExchange] = TalonExchange
    m.handlers[EventPlayAgain] = PlayAgain
}

// called in readmessage function to recognise the event
func (m *Manager) routeEvent(event Event, c *Client) error {
    handler, ok := m.handlers[event.Type]
    if ok {
        err := handler(event, c)
        if err != nil {
            return err
        }
        return nil
    } else {
        return errors.New("there is no such event type")
    }
}
```

Válasz:

Miután az esemény megtalálta a hozzá tartozó kezelőfüggvényt, és annak feldolgozása befejeződött, az eseménykezelő utolsó lépéseként egy válaszüzenetet küld vissza a szükséges kliensek felé. A válasz szintén Event formátumban kerül összeállításra. Az üzenet a megfelelő kliensekhez tartozó egress csatornára kerül elküldésre. Ezt a csatornát a backend oldalon a writeMessage goroutine figyeli, amely továbbítja az üzenetet a WebSocket kapcsolaton keresztül a frontend irányába.

ws/action.js - connect függvény (amely a beérkező üzeneteket kezeli):

```go
socket.onmessage = (ev) => {
            try {
            const eventdata = JSON.parse(ev.data);

            const   event   =   new   Event(eventdata.type,   eventdata.id,
eventdata.payload);

            …
            const returned_data = event.routeEvent();
            switch(event.type) { …
```

A frontend oldalon a backend felől érkező üzeneteket a socket.onmessage eseménykezelő fogadja. A beérkező adat JSON formátumban kerül feldolgozásra, majd az Event osztály routeEvent függvénye segítségével az alkalmazás azonosítja az esemény

típusát és értelmezi a hozzá tartozó payload adatokat. Az esemény típusa alapján a megfelelő kliensoldali logika fut le.

4.3 Docker és Docker Compose

A modern cloud-native alkalmazások körében a legelterjedtebb konténerizációs platform a Docker.**Hiba! A hivatkozási forrás nem található.**

4.3.1 A Docker architektúra

A Docker platform az alábbi főbb komponensekből tevődik össze.

- *Docker daemon:*

  Konténer managementért felelős, különböző Docker objektumokat kezel (image, konténer, hálózat, csatolt volume).

- *Docker client:*

  A felhasználó elsődleges kommunikációs felülete a daemon felé. A CLI parancsokat API-hívásokká alakítva küldi a daemonhoz.

  - *Docker objektumok:* Image: egy standardizált leíró (OCI) konténerek létrehozásához.
  - *Konténer*: egy image izoláltan futtatható pédánya.
  - *Volume*: tartós adatterület, konténer életciklusától független.
  - *Network*: kommunikációs réteg a konténerek illetve konténer és host között.

- *Docker build:*

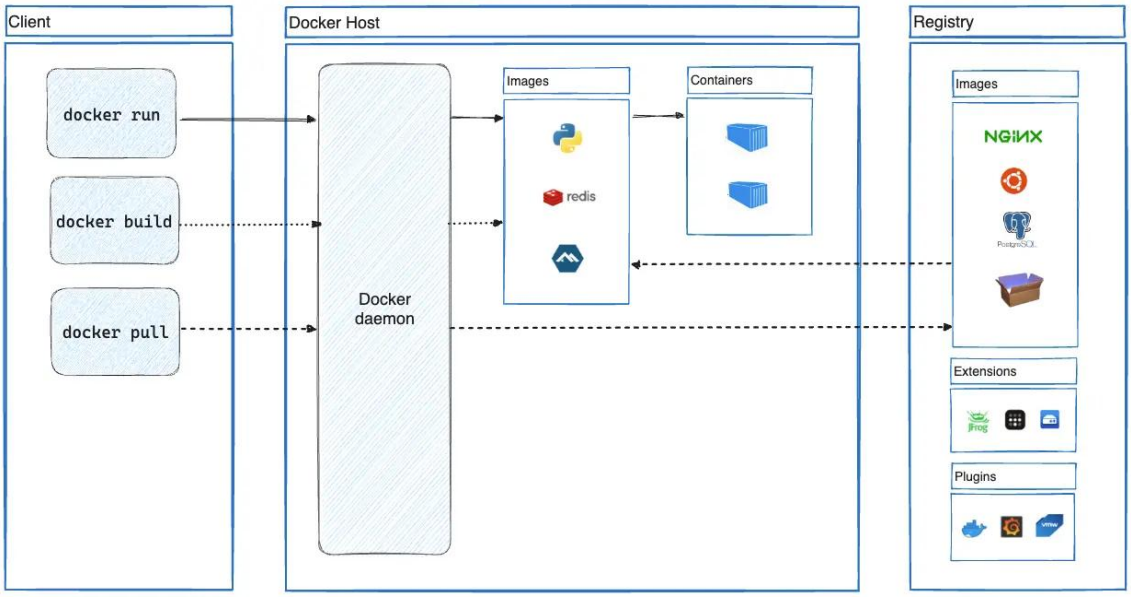
  Egy deklaratív leírást nyújt a felhasználónak konténerek létrehozásához. Ezeket a leírókat nevezzük Dockerfile-nak.

- *Docker regisrty:*

  Egy mechanizmus elkészített imagek tárolására és megosztására. Lehet privát vagy publikus (Docker Hub).

- *Docker compose:*

  Deklaratív leírást nyújt több konténeres alkalmazások konfigurációjára és együttes futtatására. Egy compose file (yaml) segítségével a felhasználó szolgáltatásokat, környezeti változókat, csatolt volumeokat, és hálózatokat definiálhat. Célja főleg fejlesztő, illetve tesztkörnyezetek létrehozásának egyszerűsítése. Éles környezetben egy komolyabb konténer-orchesztrációs[11] megoldásra van szükség, ilyen például a Kubernetes.



14. ábra A Docker architektúra

4.3.2 Dockerfileok

A backend és frontend dockerfile-ja többlépcsős build megoldást alkalmaz, ezzel elkülönítve a fejlesztői és éles környezeteket.[3]

4.3.2.1 Frontend

```
FROM node:20-alpine AS base (last pushed 2 days ago)

WORKDIR /frontend

COPY package*.json ./

RUN npm ci

COPY . .

FROM base AS dev
EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0", "--port", "5173"]

FROM base AS prod-build
RUN npm run build

FROM nginx:1.27-alpine (last pushed 8 months ago)

COPY nginx.conf /etc/nginx/conf.d/default.conf

COPY --from=prod-build /frontend/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

15. ábra Frontend Dockerfile

Az alapréteg (base) a node:20-alpine képre épül.

A dev szakasz a fejlesztési környezetet szolgálja. Itt a Vite fejlesztői szerver indul el, amely a 5173-as porton érhető el, és lehetővé teszi az alkalmazás élő újratöltését fejlesztés közben.

A prod-build szakaszban a frontend alkalmazás statikus buildje készül el az npm run build parancs segítségével.

A végső futtatási szakasz egy nginx:1.27-alpine alapképre épül, amely statikus fájlkiszolgálóként működik. Az nginx egy nagy teljesítményű webszerver, amelyet gyakran alkalmaznak statikus frontend fájlok kiszolgálására, illetve reverse-proxyként.

4.3.2.2 Backend

```
FROM golang:1.24-alpine AS builder (last pushed 2 days ago)

WORKDIR /backend

COPY go.mod go.sum ./

RUN go mod download

COPY . ./

RUN CGO_ENABLED=0 GOOS=linux go build -o /ulti-game-BE

FROM alpine:latest (last pushed 3 days ago)

WORKDIR /app

COPY --from=builder /ulti-game-BE /app/ulti-game-BE

EXPOSE 3000

CMD [ "/app/ulti-game-BE" ]
```

16. ábra Backend Dockerfile

A golang:1.24-alpine alapképre épül.

A függőségek cache-elése érdekében először azok leíró fáljait (go.mod és go.sum) másoljuk át, majd telepítjük (go mod download). A forráskód bemásolása után az alkalmazás statikusan fordítjuk (CGO_ENABLED=0) annak érdekében, hogy a létrejött bináris különböző rendszerkönyvtáraktól függetlenül futtatható legyen.

A második szakasz egy minimális alpine alapképre épül, amely kizárólag a lefordított bináris állományt tartalmazza. A konténer a 3000-es portot teszi elérhetővé, és indításkor közvetlenül a lefordított backend binárist futtatja.

4.3.3 Docker Compose felépítés

Minimum három szolgáltatás szükséges az alkalmazás lokális futtatásához: frontend, backend, és adatbázis. [1]

4.3.3.1 Frontend

A frontend szolgáltatás biztosítja az Nginx-alapú webes kiszolgálást. A konténer image-ét a frontend saját Dockerfile-ja állítja elő, amelyet a Compose a build szakaszban hivatkozik. A kommunikációhoz a 8080-as és 8443-as portokat publikálja, így a szolgáltatás HTTP és HTTPS protokollon keresztül is elérhető. A HTTPS kommunikációhoz szükséges tanúsítványok read-only módban csatolt volume-on keresztül kerülnek a konténerbe.

A frontend a backend szolgáltatástól függ (*depends_on*), ezért csak akkor indul el, amikor a backend már működőképes. A szolgáltatás *restart: unless-stopped* újraindítási stratégiát alkalmaz, amely biztosítja, hogy a konténer folyamatosan üzemeljen, és csak explicit leállítás esetén szűnjön meg a működése.

```
frontend:
  build:
    context: ./ulti-game-FRE
  image: ulti-game-frontend:latest
  container_name: ulti-frontend
  depends_on:
    - backend
  ports:
    - "8080:80"
    - "8443:443"
  volumes:
    - ./ulti-game-FRE/server.crt:/etc/nginx/certs/server.crt:ro
    - ./ulti-game-FRE/server.key:/etc/nginx/certs/server.key:ro
  restart: unless-stopped
```

4.3.3.2 Backend

A backend szolgáltatás nyújtja a frontendnek az API-t illetve valós idejű kommunikációt, és felelős az adatbázissal való kommunikációért. A konténer image-ét a projekt backend mappájában található Dockerfile határozza meg, amelyet a Compose a build szakasz segítségével épít fel. A szolgáltatás környezeti változókon keresztül kapja meg a különböző konfigurációs beállításait (DB_DRIVER, DATABASE_URL_FILE, JWT_SECRET_FILE). A DB_DRIVER környezeti változóval jelezhető a backendnek az adatbázis mögöttes implementációja. A bizalmas beállítások értékei Docker secrets mechanizmuson keresztül érkeznek, az adatok biztonságos kezelésének érdekében. A HTTPS kommunikációhoz szükséges tanúsítványok a frontendhez hasonlóan read-only módban csatolt volume-on keresztül kerülnek a konténerbe.

A backend az adatbázis szolgáltatástól függ, és csak akkor indul el amikor az elérhetővé válik (*depends_on*). A szolgáltatás elérhetőségét a Compose *healthcheck* attribútumával adjuk meg, amely a szolgáltatás saját /health végpontját kérdezi le, majd sikeres válasz esetén a konténer elérhetőnek számít. A szolgáltatás a frontendhez hasonlóan *restart: unless-stopped* újraindítási stratégiát alkalmazza.

```
backend:
  build:
    context: ./ulti-game-BE
  image: ulti-game-backend:latest
  container_name: ulti-backend
  environment:
    DB_DRIVER: postgres
```

```
      DATABASE_URL_FILE: /run/secrets/database_url
      JWT_SECRET_FILE: /run/secrets/jwt_secret
    secrets:
      - database_url
      - jwt_secret
    volumes:
      - ./ulti-game-BE/server.crt:/app/server.crt:ro
      - ./ulti-game-BE/server.key:/app/server.key:ro
    depends_on:
      postgres:
        condition: service_healthy

    restart: unless-stopped

    healthcheck:
      test: ["CMD", "curl", "-kf", "https://localhost:3000/health" ]
      interval: 10s
      timeout: 3s
      retries: 10
```

4.3.3.3 Adatbázis

Az alkalmazás backend komponense a loose-coupling cloud-native architekúrális elvek érdekében több adatbázis megoldást is támogat. A project jelenlegi állapota PostgreSQL és MySQL adatbázisokat támogat. A két adatbázis szolgáltatást külön compose fileok definiálják *docker-compose.postgres.yml* illetve *docker-compose.mysql.yml*.

Mindkét adatbázis esetében hasonló a konfiguráció. A jelszavak a backend bizalmas adataihoz hasonlóan Docker secrets által vannak kezelve. Úgyszint a backendhez hasonlóan mindkét adatbázis szolgáltatásnak *healthcheck*-ek vannak definiálva.

MySQL:

```
  mysql:
    image: mysql:8.0
    container_name: ulti-mysql
    environment:
      MYSQL_DATABASE: ulti
      MYSQL_USER: ulti
      MYSQL_PASSWORD_FILE: /run/secrets/mysql_password
      MYSQL_ROOT_PASSWORD_FILE: /run/secrets/mysql_root_password
    secrets:
      - mysql_password
      - mysql_root_password
    ports:
      - "3306:3306"
    volumes:
      - mysqldata:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -u$${MYSQL_USER} -
p$${MYSQL_PASSWORD} --silent"]
```

```
      interval: 5s
      timeout: 3s
      retries: 20
```

PostgreSQL:

```
  postgres:
    image: postgres:16-alpine
    container_name: ulti-postgres
    environment:
      POSTGRES_DB: ulti
      POSTGRES_USER: ulti
      POSTGRES_PASSWORD_FILE: /run/secrets/postgres_password
    secrets:
      - postgres_password
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
          test:  ["CMD-SHELL",  "pg_isready  -U  $${POSTGRES_USER}  -d
$${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 20
```

4.3.3.4 Secrets

A bizalmas konfigurációs értékek Docker secrets segítségével vannak kezelve. Ez a mechanizmus csak olvasható módon biztosítja a futó konténerek számára a szükséges bizalmas adatokat, így azok nem kerülnek a verziókezelésbe vagy környezeti változókba. A Compose konfigurációban az alábbi módon vannak definiálva:

```
secrets:
  postgres_password:
    file: ./secrets/postgres_password.txt
  database_url:
    file: ./secrets/database_url.postgres.txt
  jwt_secret:
    file: ./secrets/jwt_secret.txt
  mysql_password:
    file: ./secrets/mysql_password.txt
  mysql_root_password:
    file: ./secrets/mysql_root_password.txt
```

Ez a megoldás egységes és biztonságos titok kezelést biztosít, lehetővé téve, hogy a különböző környezetek saját secret fájlokat használjanak anélkül, hogy a Compose konfigurációt módosítani kellene.

5 Eredmény

Ebben a fejezetben bemutatom, hogy a *cloud-native* elvek alkalmazása érdekében milyen megoldások kerültek implementálásra, valamint ismertetem azokat a módszereket és fejlesztési irányokat, amelyekkel az alkalmazás tovább fejleszthető.

5.1 Megvalósított Cloud Native Patternek

5.1.1 Konténerizáció

A webalkalmazás fejlesztése során a konténerizáció megvalósításra került, amelynek keretében külön Dockerfile készült a frontend és a backend alkalmazás számára. Ezek a Dockerfile-ok pontosan definiálják az egyes komponensek futtatási környezetét, függőségeit és indítási módját, ezáltal biztosítva az egységes és reprodukálható működést különböző környezetekben.

A frontend és a backend konténerek együttes kezelésére Docker Compose konfiguráció készült, amely deklaratív módon írja le az alkalmazást, beleértve az egyes szolgáltatások közötti kapcsolatokat, a szükséges környezeti változókat, valamint az indítási sorrendet. Ez lehetővé teszi az alkalmazás egyszerű indítását, leállítását és újraindítását egyetlen parancs segítségével.

A konténerizáció egyik legnagyobb előnye, hogy az alkalmazás egyes komponensei független példányokban futtathatók. Ez megteremti a horizontális skálázás alapját

5.1.2 Állapotmentesség

A webalkalmazás esetében a WebSocket kapcsolat kivételével minden komponens stateless módon működik. Ez cloud-native szempontból kifejezetten előnyös, mivel lehetővé teszi a szolgáltatások horizontális skálázását, egyszerű újraindítását, valamint a terhelés hatékony elosztását több példány között. Amennyiben egy backend példány kiesik, egy másik példány képes azonnal átvenni a feladatát, mivel nem támaszkodik lokálisan tárolt állapotra. A megvalósított WebSocket alapvetően stateful kapcsolat.

Egy Redis vagy más elosztott cache megoldás alkalmazásával a WebSocket kapcsolatokhoz tartozó állapotok is leválaszthatóak lesznek.

5.1.3 Külső konfiguráció

A külső konfiguráció azt jelenti, hogy az alkalmazás működéséhez szükséges konfigurációs adatok nem a kódban vannak rögzítve, hanem futtatási időben külső forrásból kerülnek betöltésre. Ilyen konfigurációk például: adatbázis elérési út. A konfiguráció változása nem igényel beavatkozást a kódba.

Az alkalmazás a cloud-native szemléletnek megfelelően a konfigurációt teljes mértékben leválasztja a kódtól. A futtatási paraméterek környezeti változókon és Docker secreteken keresztül kerülnek átadásra, ami lehetővé teszi az egységes build használatát különböző környezetekben, növeli a biztonságot, valamint megkönnyíti az üzemeltetést és a skálázást.

A bemutatott webalkalmazás esetében a külső konfiguráció több szinten is megvalósul. A backend alkalmazás a működéséhez szükséges beállításokat környezeti változókon keresztül kapja meg, például az adatbázis-kezelő típusát vagy a titkos kulcsok elérési útját. Az érzékeny adatok, mint az adatbázis elérési URL-je vagy a JWT token aláírásához használt kulcs, a secrets segítségével kerülnek átadásra, így ezek nem részei sem a forráskódnak, sem a Docker image-nek. Ez a megoldás növeli a biztonságot, és megfelel a cloud-native alkalmazásokkal szemben támasztott elvárásoknak.

5.1.4 Health checks

Cloud-native környezetben egy szolgáltatás futtatása önmagában nem garancia arra, hogy ténylegesen képes forgalmat kiszolgálni. A health check célja, hogy a rendszer automatikusan meg tudja állapítani él-e a folyamat, kész-e kérések fogadására, és szükség esetén tudjon újraindítást / forgalomból kivonást kezdeményezni.

Ez a módszer közvetlenül támogatja a rendszer ellenállóképesség menedzselhetőség.

A Docker Compose konfigurációban alkalmazott health checkek és restart policy-k hozzájárulnak a rendszer ellenállóképességéhez. Amennyiben egy konténer hibás állapotba kerül, a rendszer képes azt automatikusan újraindítani. Ez biztosítja, hogy részleges meghibásodás esetén is fennmaradjon a szolgáltatás elérhetősége.

A WebSocket kapcsolat egy hosszú életű kommunikációs csatorna, ezért külön figyelmet igényel annak felügyelete. Cloud-native környezetben fontos annak biztosítása, hogy a kapcsolat valóban aktív maradjon, és időben felismerhető legyen a kapcsolat megszakadása.

client.go-ban lévő Heartbeat set up:

```
var (
    //how much we wait before the connection drops
    pongWait = 10 * time.Second

    // how often we send pings
    pingInterval = (pongWait * 9) / 10
)

// Resetting timer
func (c *Client) pongHandler(pongMsg string) error {
    log.Println("pong")
    return c.connection.SetReadDeadline(time.Now().Add(pongWait))
}
```

client.go readMessage goroutine-ban lévő Heartbeat:

```
if err := c.connection.SetReadDeadline(time.Now().Add(pongWait)); err != nil
{
        log.Println(err)
        return
    }

c.connection.SetPongHandler(c.pongHandler)
```

Ezt a célt szolgálja az úgynevezett heartbeat mechanizmus, amely során a kliens vagy a szerver rendszeres időközönként egy kis méretű „életjel” üzenetet küld. Amennyiben egy meghatározott időn belül nem érkezik válasz, a kapcsolat megszakadtnak tekinthető.

5.1.5 Graceful shutdown

Konténerizált környezetben az alkalmazás példányai gyakran újraindulnak ezért fontos, hogy a folyamat eldobható legyen. Gyorsan indítható és szabályosan leállítható. A graceful shutdown kódod ezt a célt szolgálja

Ez a megoldás a cloud-native rendszerekben elvárt ellenállóképesség és menedzselgetőség tulajdonságokat erősíti, mivel a rendszer jól viselkedik leálláskor, és jobban együttműködik az üzemeltetési környezettel.

main.go fájlban implementált graceful shutdown:

```
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit
log.Println("Shutting down server...")

// Create context with timeout for graceful shutdown
ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()

if err := srv.Shutdown(ctx); err != nil {
    log.Fatalf("Server forced to shutdown: %s", err)
}

// Close database connections
if database.Database != nil {
    _ = database.Database.Close()
}
```

5.1.6 Adatbázis interfész

Az adatbázis-interfész bevezetésének célja, hogy az alkalmazás adatkezelési rétege függetlenné váljon a konkrét adatbázis-kezelő rendszertől. Egy ilyen absztrakciós réteg lehetővé teszi, hogy az alkalmazás üzleti logikája ne függjön közvetlenül sem a MySQL, sem a PostgreSQL specifikus megvalósítási részleteitől.

Ennek a megközelítésnek egyik legfontosabb előnye a lazán csatolt felépítés (loose coupling), mivel az adatbázis-kezelő cseréje vagy módosítása nem igényel változtatást az alkalmazás magasabb szintű logikájában.

További előny, hogy az alkalmazás rugalmasabban konfigurálható különböző környezetekben. Például fejlesztési vagy tesztelési környezetben eltérő adatbázis-kezelő használható, mint éles környezetben, anélkül, hogy a forráskód módosítására lenne szükség. Ez jól illeszkedik a cloud-native szemlélethez, ahol az alkalmazások gyakran eltérő infrastruktúrán futnak.

Az adatbázis-interfész alkalmazása emellett növeli a rendszer karbantarthatóságát és bővíthetőségét. Új adatbázis-kezelő rendszer támogatása a későbbiekben külön implementációként hozzáadható, miközben az alkalmazás többi része változatlan marad.

5.2 Tovább fejlesztési lehetőségek

5.2.1 Redis

Ez a megközelítés több cloud-native előnyt is biztosít. A backend szolgáltatások stateless módon működhetnek, mivel az alkalmazás állapota nem egyetlen szerver memóriájához kötődik. Ennek köszönhetően a WebSocket kapcsolatok több backend példány között is eloszthatók, ami lehetővé teszi a rendszer horizontális skálázását terhelésnövekedés esetén. Az állapot külső komponensbe történő kiszervezése csökkenti az egyetlen szerverhez kötött működésből adódó hibák kockázatát, és növeli a rendszer ellenállóképességét. Egy Redis-alapú pub/sub vagy cache mechanizmus alkalmazásával az események továbbítása és a játékállapot szinkronizálása több backend példány között is hatékonyan megvalósítható lenne.

5.2.2 Logging

Az alkalmazás jelenlegi megvalósításában mind a frontend, mind a backend oldalon alapvető naplózási mechanizmusok kerülnek alkalmazásra. A frontendben a console.log, míg a backendben a Go beépített log csomagjának függvényei szolgálnak az események és hibák naplózására. Ezek a megoldások a standard kimenetre írják a logokat, ami összhangban van a cloud-native alkalmazások alapvető naplózási elvével. Ugyanakkor ez a megközelítés elsősorban fejlesztési célokat szolgál, és nem biztosít strukturált, szintalapú naplózást. A rendszer továbbfejlesztéseként érdemes lenne egy egységes, strukturált logging megoldást bevezetni, amely jobban támogatja a megfigyelhetőséget és az üzemeltetést cloud-native környezetben.

A logging a cloud-native rendszerek egyik alapvető megfigyelhetőségi eszköze, amely az alkalmazás működése során keletkező események, állapotváltozások és hibák rögzítését jelenti. Cloud-native környezetben a logok elsődleges célja nem csupán a hibakeresés, hanem az alkalmazás viselkedésének folyamatos nyomon követése, különösen olyan rendszerek esetében, ahol az alkalmazás több példányban, dinamikusan indul és áll le.

5.3 Konklúzió

A szakdolgozat során egy teljes értékű webalkalmazás valósult meg, amely a frontend és backend komponensek együttes fejlesztését igényelte. A projekt lehetőséget adott arra, hogy a modern webes technológiák gyakorlati alkalmazása mellett a cloud-native pattern-s alapelvei is megismerjem.

A fejlesztés tapasztalatai alapján elmondható, hogy cloud-native módon webalkalmazást készíteni egyszerre jelent előnyt és kihívást. A rendelkezésre álló eszközök és minták széles választéka nagy rugalmasságot biztosít, ugyanakkor tudatos tervezést igényel annak eldöntése, hogy mely megoldások alkalmazása indokolt egy adott projekt esetében. A cloud-native elemek közül vannak egyszerűen implemetálhatóak (konténerizáció, a külső konfiguráció vagy az alapvető health checkek). Már ezek alkalmazása esetén egy kisebb rendszer esetében is előnyökkel jár. Ezek használata egy modern webalkalmazás fejlesztése során alapvető követelménynek tekinthető.

A projekt rávilágított arra is, hogy a Docker és a hozzá kapcsolódó technológiák ismerete kulcsfontosságú a cloud-native alkalmazások fejlesztésében, mivel ezek biztosítják az egységes futtatási környezetet, a reprodukálhatóságot és a skálázhatóság alapjait.

Összességében a megvalósított alkalmazás jó alapot teremt a cloud-native szemlélet további elmélyítésére, és megfelelő kiindulópontként szolgál a rendszer későbbi bővítéséhez és továbbfejlesztéséhez.

6 Irodalomjegyzék

[1] S. Deng et al., "*Cloud-Native Computing: A Survey From the Perspective of Services*" in Proceedings of the IEEE, vol. 112, no. 1, pp. 12-46, Jan. 2024

[2] Y. Hongyu and W. Anming, "*Migrating from Monolithic Applications to Cloud Native Applications*" 2023 8th International Conference on Computer and Communication Systems (ICCCS), Guangzhou, China, 2023, pp. 775-779

[3] P. P. T and C. Kumar, "*Building cloud native application — analysis for multi-component application deployment*" 2021 International Conference on Computer Communication and Informatics (ICCCI), Coimbatore, India, 2021, pp. 1-6

[4] P. Poudel, "*What is a Single-Page App?"* Beginning Elm, 2018. [Online]. Available: https://elmprogramming.com/what-is-a-single-page-app.html. [Accessed: Dec. 21, 2025]

[5] Matthew A. Titmus.: *Cloud Native GO, Building Reliable Services in Unreliable Environments* (2021 April)

[6] Vue.js Team, "*Vue Router Guide*," *Vue Router Documentation*. [Online]. Available: https://router.vuejs.org/guide/. [Accessed: Dec. 21, 2025].

[7] Vite Team, "*Vite Guide*," *Vite Documentation*. [Online]. Available: https://vite.dev/guide/. [Accessed: Dec. 21, 2025].

[8] Vue.js Team, "*Introduction to Vue*," *Vue.js Documentation*. [Online]. Available: https://vuejs.org/guide/introduction.html. [Accessed: Dec. 21, 2025].

[9] The Go Authors, "*The Go Programming Language*," *go.dev*. [Online]. Available: https://go.dev. [Accessed: Dec. 21, 2025].

[10] DigitalOcean, "*Horizontal Scaling vs Vertical Scaling*," DigitalOcean Resources. [Online]. Available: https://www.digitalocean.com/resources/articles/horizontal-scaling-vs-vertical-scaling. [Accessed: Dec. 21, 2025].

[11] Linux Professional Institute, "*Containers and Container Orchestration*," LPI Learning Materials. [Online]. Available: https://learning.lpi.org/en/learning-materials/701-200/701/701.1/701.1_02/. [Accessed: Dec. 21, 2025].

[12] Gin Gonic — *Web framework documentation*, Gin Gonic. [Online]. Available: https://gin-gonic.com

[13] *Websocket vs WebRTC* — FAQ, Svix Resources. [Online]. Available: https://www.svix.com/resources/faq/websocket-vs-webrtc/

[14] *Server-Sent Events vs WebSockets* — freeCodeCamp article, freeCodeCamp. [Online]. Available: https://www.svix.com/resources/faq/websocket-vs-webrtc

[15] *Docker Documentation*, Docker. [Online]. Available: https://docs.docker.com

7 Függelék

7.1 I. Függelék Nyilatkozat generatív mesterséges intelligencia alkalmazásáról

☐ **Nem használtam** semmilyen generatív MI segédeszközt.

☐ **Használtam** generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

Felhasználási módok	**Generatív MI eszköz(ök) neve**	**Érintett részek** (fejezet, oldalszám, hivatkozás)	**Használat becsült aránya** (felhasználási módonként)
Irodalomkutatás			
Prompt lényegi része			
Programkód generálása	ChatGPT	kód	10%
Prompt lényegi része	Check for errors and syntax mistakes		
Új ötletek, megoldási javaslatok generálása			
Prompt lényegi része			
Vázlat létrehozása (szövegstruktúra, vázlatpontok)			
Prompt lényegi része			
Szövegblokkok létrehozása			
Prompt lényegi része			
Képek generálása illusztrációs célból			
Prompt lényegi része			

Adatvizualizáció, grafikonok generálása adatpontok alapján			
Prompt lényegi része			
Prezentáció készítése			
Prompt lényegi része			
Egyéb – Már meglévő szöveg javítása	ChatGPT		15%
Prompt lényegi része	Kérlek segíts átfogalmazni egy részt, hogy szakdolgozathoz illő legyen, ne egészísd ki		
Összesített százalékos érték (a feladat érdemi részére nézve)			**5%**
Összesített érték rövid, szöveges indoklása: **AI eszközöket főképp hibakeresésre használtam, illetve ahhoz használtam, hogy a szakdolgozat szebben legyen megfogalmazva. Dolgozat tartalmát nem változtattam, a kódban pedig próbáltam minél kevesebbszer támaszkodni rá.**			